

Содержание

Аннотация	4
Ключевые слова	4
Введение	5
Описание предметной области	5
Цели и задачи	5
Результаты работы и их новизна	6
Структура всей работы	7
Разделение задач по участникам	8
1 Обзор литературы	9
1.1 Классические методы прогнозирования финансовых временных рядов	9
1.2 Методы глубокого обучения для финансовых рядов	9
1.2.1 Бинарные модели рыночных режимов и их ограничения	9
1.3 Прогнозирование на российском фондовом рынке	10
1.4 Обучение представлений и кластеризация временных рядов	10
1.5 Оптимизация портфеля	10
1.6 Позиционирование данной работы	11
2 Описание общего пайплайна	12
3 Работа с данными	14
3.1 Данные	14
3.2 Предобработка данных и создание признаков	14
4 Методы построения эмбедингов	16
4.1 TS2Vec Embedder	16
4.2 Handmade Embedder	17
5 Кластеризация эмбедингов	18
5.1 KMeans	19
5.1.1 Визуализация и анализ кластеров	19
5.1.1.1 Метрики анализа кластеров	19
5.1.1.2 TS2Vec	20
5.1.1.3 Handmade	23
5.1.1.4 Сравнение TS2Vec и Handmade	25
5.2 HDBSCAN	26
6 Модели прогнозирования доходности	26
6.1 Подбор гиперпараметров	26
6.2 ARIMA	27
6.2.1 Архитектура	27
6.3 Реализация	27
6.4 MR-AR	28
6.4.1 Архитектура	28
6.4.2 Реализация	28
6.5 LightGBM	29

6.5.1	Архитектура	29
6.5.2	Реализация	30
6.6	LSTM	31
6.6.1	Архитектура	31
6.6.2	Реализация	32
6.7	GRU	33
6.7.1	Архитектура	33
6.7.2	Реализация	33
6.8	CNN	34
6.8.1	Архитектура	34
6.8.2	Реализация	34
6.9	Transformer	36
6.9.1	Архитектура	36
6.9.2	Реализация	36
6.10	Метрики оценки	37
7	Методология тестирования	39
7.1	Общая схема эксперимента	39
7.2	Подготовка данных и скользящее окно	39
7.3	Построение эмбедингов и кластеризация	39
7.4	Обучение специализированных моделей	39
7.5	Бэктестирование торговой стратегии	40
7.6	Портфельное тестирование	41
8	Результаты	41
8.1	Оценка качества предсказательной способности моделей	41
8.2	Модели без кластеризации	41
8.3	Модели с кластеризацией	43
8.3.1	TS2Vec + KMeans	43
8.3.1.1	C0 — нейтральный (боковое движение)	44
8.3.1.2	C1 — бычий тренд	44
8.3.1.3	C2 — высокая волатильность	45
8.3.1.4	C3 — медвежий тренд	46
8.3.1.5	C4 — аномальные движения	46
8.3.2	Handmade + KMeans	47
8.3.2.1	C0 — нейтральный (боковое движение)	47
8.3.2.2	C1 — бычий тренд	48
8.3.2.3	C2 — нисходящий с отрицательной асимметрией	49
8.3.2.4	C3 — кризисный режим	49
8.3.3	Промежуточные итоги и выводы	50
	Список литературы	51

Аннотация

В работе рассматривается задача прогнозирования доходности акций на Московской бирже (MOEX) [1], [2] и формирования на этой основе инвестиционного портфеля [3]. В ходе исследования проведён сравнительный анализ моделей предсказания доходности: LSTM [4], GRU [5], CNN [6], Transformer [7], ARIMA [8], MR-AR [9], LightGBM [10]. Реализованы также собственные методы с использованием эмбедингов временных рядов [11] и кластеризации [12].

Оказалось, что наиболее эффективными оказались алгоритмы, разбивающие окна акций на кластеры и предсказывающие доходность в зависимости от кластера.

Помимо этого, для каждого алгоритма была построена оптимизация портфеля по Марковицу и проведён бэктестинг на исторических данных [13].

Ключевые слова

прогнозирование цен, Московская биржа, формирование портфеля, алгоритмическая торговля, бэктестирование, машинное обучение, эмбединги, кластеризация

Введение

Описание предметной области

Прогнозирование цен акций — одна из центральных задач количественных финансов, привлекающая внимание как академического сообщества, так и практикующих участников рынка. Российский фондовый рынок (Московская биржа, MOEX) отличается рядом специфических черт: высокой концентрацией в сырьевых и финансовых секторах, неравномерной ликвидностью инструментов, чувствительностью к геополитическим факторам, — что делает задачу прогнозирования ценовой динамики особенно нетривиальной.

Традиционные подходы — технический анализ, классические модели временных рядов (ARIMA) — слабо адаптируются к смене рыночных режимов. Методы глубокого обучения (LSTM, Transformer, CNN) повышают качество прогноза, однако, как правило, обучаются независимо для каждого инструмента и не используют межинструментальную информацию о схожести ценовых паттернов. Ещё одна системная проблема — нестационарность: паттерны, эффективно работавшие в прошлом для конкретной бумаги, со временем деградируют. При этом аналогичные паттерны могут проявляться в других инструментах или в других временных периодах.

Настоящая работа предлагает альтернативный подход. Вместо прямого предсказания цены или доходности для каждого инструмента отдельно мы сначала отображаем скользящие окна ценовых рядов всех акций в единое латентное пространство (получаем эмбединги), после чего кластеризуем эти представления и используем кластерную структуру как основу для адаптивного прогнозирования и оптимизации портфеля по Марковицу. Гипотеза состоит в том, что близость в латентном пространстве отражает содержательное сходство ценовой динамики вне зависимости от конкретного инструмента и момента времени — и именно этот перенос знаний между инструментами и периодами позволяет строить более робастные торговые стратегии.

Цели и задачи

1. Рассмотреть базовые алгоритмы предсказания временных рядов (LSTM, GRU, CNN, Transformer, ARIMA, MR-AR и LightGBM), сравнить их и выбрать лучшие на данных MOEX.
2. Разработать свои собственные подходы к предсказанию доходностей акций на Московской бирже.
3. Провести сравнительный анализ всех моделей по метрикам качества
4. Построить оптимизацию портфеля по Марковицу и провести бэктестирование на исторических данных MOEX.

В качестве базовых моделей сравнения рассматриваются: LSTM, GRU, CNN, Transformer, ARIMA, MR-AR и LightGBM — каждая из которых имеет свои преимущества и недостатки в контексте финансовых временных рядов.

Фундаментальное ограничение большинства существующих подходов состоит в том, что модель прогнозирования обучается на всей совокупности данных сразу, усредняя поведение рынка через все режимы и инструменты. Такая модель вынуждена искать глобальные закономерности, жертвуя точностью на локальных паттернах. Между тем рынок нестационарен: паттерн, новый для конкретной бумаги в данный момент, вполне

мог встречаться ранее в других акциях или в других временных периодах. Иными словами, перенос знаний между инструментами и периодами потенциально ценен, но стандартными методами не используется.

Предлагаемый в работе подход строится на следующей идее. Весь массив ценовых рядов рассматривается как единая совокупность скользящих окон фиксированной длины. Каждое окно отображается в латентное пространство — получается эмбединг, компактно описывающий форму ценового движения. Близкие в этом пространстве окна образуют кластеры, соответствующие схожим рыночным режимам или каким-то определённым паттернам. Для каждого кластера затем выбирается и обучается наилучшая прогнозная модель, способная более точно уловить специфику именно этого типа динамики.

Ключевым шагом является качественное извлечение признаков из окон: эмбединги должны сближать окна с похожей динамикой и разводить окна с различным поведением. Применение эмбедингов позволяет, во-первых, абстрагироваться от масштаба цен отдельных акций, а во-вторых, выявлять повторяющиеся паттерны поведения, которые могут соответствовать определённым рыночным режимам или фазам движения. В работе рассматриваются и сравниваются два принципиально разных подхода к построению эмбедингов:

- **Обучаемые эмбединги (TS2Vec)** — нейросетевой кодировщик (контрастивное обучение) [14] выявляет скрытые межоконные зависимости, которые могут быть неочевидны или вовсе неизвестны заранее.
- **Рукотворные эмбединги (Handmade)** — векторы из интерпретируемых статистических признаков окна (волатильность, наклон тренда, асимметрия и др.), не требующие обучения модели.

После того как эмбединги построены, можно уже использовать кластеризацию. Таким образом, мы получаем разбиение окон всех акций на кластеры, которые соответствуют схожим паттернам.

Результаты работы и их новизна

В ходе работы был разработан и реализован пайплайн, включающий обучение эмбедингов ценовых окон, кластеризацию рыночных паттернов, кластерно-адаптивное прогнозирование доходности и оптимизацию портфеля по Марковицу с бэктестированием на данных МОЕХ. При этом подтвердилась гипотеза о том, что модели, заточенные под свой конкретный кластер, работают лучше, чем модели, обученные на всех данных сразу.

Этот результат позволяет строить прогнозные системы, явным образом учитывающие текущие рыночные паттерны (не просто режимы). Это повышает устойчивость портфельных стратегий к смене паттерна рынка и снижает зависимость от глобальной стационарности данных — свойство особенно важное для российского рынка с его высокой чувствительностью к внешним шокам. Таким образом, теперь нам достаточно понять, к какому кластеру относится окно конкретной акции, чтобы сделать качественный прогноз.

Научная польза и новизна проекта заключаются в том, что он предлагает новый подход к прогнозированию цен акций, используя специально обученную модель под выделенные паттерны рынка, что позволяет более точно прогнозировать цены акций.

Актуальность исследованию придаёт тот факт, что вся работа была проделана на акциях Московской биржи, для которой написано сравнительно мало статей подобного типа.

Весь код, использованный в работе, находится в репозитории: <https://github.com/Coftochka/CourseBDAMI3>.

Структура всей работы

В Раздел 1 представлен обзор релевантной литературы по теме прогнозирования цен акций и формирования портфеля по Марковицу, а также проведено сравнение с нашей работой. Раздел 2 описывает общий пайплайн работы и его этапы, которые разделены по следующим разделам:

- Раздел 3 (работа с данными): описываются типы данных, сколько их всего, создание новых фичей, а также рассказывается, как данные разбиваются на обучающую, валидационную и тестовую выборки.
- Раздел 4 (методы построения эмбеддингов): описываются два метода построения эмбеддингов: TS2Vec (какого размера и как обучаются) и Handmade (какие фичи формируются по окну).
- Раздел 5 (кластеризация эмбеддингов): описываются два метода кластеризации: k-means и hdbscan. Как они применялись на эмбеддингах, интерпретация кластеров, а также рассказывается о неудачном эксперименте с hdbscan.
- Раздел 6 (модели прогнозирования доходностей): детально описывает базовые модели, подбор и перебор каких гиперпараметров осуществлялся и как.

Раздел 7 описывает методологию тестирования, которая была применена к выбранным моделям. В Раздел 8 детально описываются результаты работы. Рассказывается о том, какие метрики получились у моделей на каждом кластере, а также какие метрики получились у моделей без кластеризации. Проводится сравнение всех моделей по всем метрикам. А также описывается, с какими гиперпараметрами запускались модели.

Разделение задач по участникам

1. Борис Барсуков:

- Получение данных с Московской биржи (MOEX), работа с БД
- Обучение всех базовых моделей и проведение анализа
- Оптимизация портфеля по Марковицу
- Проведение бэктестирования всех алгоритмов на исторических данных

2. Шадрин Андрей:

- Построение TS2Vec эмбедингов
- Кластеризация эмбедингов (k means, hdbscan)
- Интерпретация получившихся кластеров
- На получившихся кластерах провести обучение и подбор гиперпараметров базовых моделей
- Выявить лучшую модель на каждом кластере

3. Климов Демьян:

- Построение Handmade эмбедингов, выбор статистик
- Кластеризация эмбедингов (k means, hdbscan)
- Интерпретация получившихся кластеров
- На получившихся кластерах провести обучение и подбор гиперпараметров базовых моделей
- Выявить лучшую модель на каждом кластере

1 Обзор литературы

1.1 Классические методы прогнозирования финансовых временных рядов

Задача прогнозирования цен акций привлекает исследователей на протяжении десятилетий. Одним из первых и наиболее изученных подходов остаётся семейство ARIMA-моделей [8]: они хорошо описывают краткосрочную линейную автокорреляцию доходностей и не требуют вычислительных ресурсов. Однако линейность является фундаментальным ограничением: нелинейные режимы рынка — резкие развороты, скачки волатильности, панические распродажи — ARIMA не улавливает по построению. Попытку преодолеть нестационарность представляют Markov Switching модели [9], [15]: скрытая цепь Маркова переключает AR-параметры между режимами, что лучше отражает смену рыночных фаз. Тем не менее MS-AR остаётся линейной внутри каждого режима и обучается независимо для каждого инструмента.

1.2 Методы глубокого обучения для финансовых рядов

С развитием архитектур глубокого обучения исследователи начали применять их к финансовым временным рядам. Работа [16] показала, что LSTM статистически значимо превосходит случайный лес по точности направления движения цены на данных S&P 500. Систематический обзор [17] охватывает более 150 публикаций 2005–2019 годов и констатирует, что CNN, LSTM и их гибриды устойчиво обходят классические эконометрические модели по MAE и RMSE на большинстве бенчмарков. Сравнение ARIMA, LSTM и BiLSTM [18] подтверждает преимущество нейросетевых методов на коротких горизонтах прогноза. В задаче статистического арбитража на S&P 500 [19] глубокие нейронные сети и деревья с бустингом (LightGBM-класс) показали сопоставимое качество, что обосновывает включение обоих классов в сравнительный анализ.

Архитектура Transformer [7], изначально разработанная для NLP, нашла применение в длинных временных рядах благодаря механизму внимания [20]. Вместе с тем работа [21] ставит под сомнение преимущество Transformer перед линейными моделями на стандартных бенчмарках прогнозирования, указывая на склонность к переобучению при малом числе обучающих примеров — что особенно актуально для российского рынка с ограниченной историей торгов.

1.2.1 Бинарные модели рыночных режимов и их ограничения

Отдельный класс работ опирается на идею деления рынка на два глобальных состояния — бычье и медвежье. Классическая статья [22] предложила формализовать это разделение через двухрежимную Markov Switching модель, обученную на индексных доходностях: в «бычьем» режиме ожидаемая доходность высокая и волатильность умеренная, в «медвежьем» — наоборот. Авторы показали, что такая разметка согласуется с интуитивными представлениями о состоянии рынка и позволяет улучшить качество портфельных решений. Схожие двух- и трёхрежимные конструкции применялись и в последующих работах по управлению рисками и тактической аллокации активов [15].

Тем не менее подход «бычий/медвежий» страдает от принципиального ограничения: он работает на уровне рыночного индекса и присваивает **один** режим всему рынку в данный момент времени. Реальное поведение отдельных акций куда богаче двух состояний:

наряду с направленными трендами встречаются боковая консолидация, высоковолатильные пилообразные движения, V-образные развороты, плавные восходящие каналы с низкой волатильностью и многие другие паттерны, каждый из которых требует собственной прогнозной стратегии. Кроме того, два инструмента, одновременно находящиеся в «бычьем» рыночном режиме, могут демонстрировать совершенно разную ценовую динамику на уровне отдельного окна.

Предлагаемый нами подход принципиально отличается: вместо бинарной глобальной метки мы строим эмбединги коротких ценовых окон каждого инструмента и кластеризуем их в K групп, выученных непосредственно из данных. Получаемые кластеры не ограничены двумя полюсами — они отражают всё многообразие локальных ценовых паттернов, характерных именно для акций МОЕХ. Это позволяет обучить специализированную прогнозную модель для каждого типа динамики, а не усреднять поведение рынка через единую «бычью» или «медвежью» фазу.

Общий изъян перечисленных подходов: все они обучают отдельную модель на каждом инструменте (или на всей выборке без учёта локальных режимов), не используя межинструментальную структуру ценовых паттернов.

1.3 Прогнозирование на российском фондовом рынке

Применение машинного обучения к данным Московской биржи остаётся значительно менее изученным, чем для западных рынков. Работа [1] сравнивает ряд классических алгоритмов МО на российских акциях и фиксирует, что ансамблевые методы превосходят линейные регрессии, однако охватывает лишь небольшую выборку ликвидных бумаг и не учитывает look-ahead bias при отборе инструментов. Исследование [2] тестирует LSTM и градиентный бустинг на внутридневных данных МОЕХ и подтверждает превосходство нейросетей на волатильных периодах, однако оптимизация портфеля в работе не рассматривается.

Таким образом, комплексных систем, объединяющих широкий набор моделей, кросс-инструментальное обучение эмбедингов и портфельную оптимизацию именно на данных МОЕХ, в литературе представлено крайне мало.

1.4 Обучение представлений и кластеризация временных рядов

Идея получать компактные векторные представления (эмбединги) временных рядов активно развивается в последние годы. Обзор [12] систематизирует подходы к кластеризации временных рядов за десятилетие и выделяет два основных направления: методы на основе сырых данных (DTW, евклидово расстояние) и методы на основе признаков представлений. Самообучающийся подход TNC [14] обучает кодировщик так, чтобы временные соседи давали близкие представления, — что концептуально близко к нашему контрастивному обучению. Архитектура TS2Vec [11], применённая в данной работе, устанавливает state-of-the-art на ряде задач классификации и регрессии временных рядов за счёт иерархического контрастивного обучения по временным шкалам.

Применение эмбедингов к кластеризации рыночных режимов с последующим адаптивным прогнозом на данных МОЕХ в существующей литературе не представлено.

1.5 Оптимизация портфеля

Классическая mean-variance оптимизация [3] страдает от нестабильности весов при плохо обусловленной ковариационной матрице [23]. Регуляризация по Ледуа–Вольфу [24] суще-

ственно снижает ошибку оценки ковариаций на малых выборках. Тем не менее качество оптимизации критически зависит от точности прогноза ожидаемых доходностей — именно здесь кластерно-адаптивный прогноз, предлагаемый в настоящей работе, создаёт прямое конкурентное преимущество.

1.6 Позиционирование данной работы

Анализ литературы позволяет выделить четыре ключевых пробела, которые заполняет настоящее исследование.

Во-первых, работы по российскому рынку [1], [2] ограничиваются небольшим числом ликвидных инструментов и не решают задачу кластеризации режимов. Наш подход охватывает весь торгуемый рынок MOEX без ручного отбора акций, что исключает survivorship bias.

Во-вторых, глубокообучающиеся модели [16], [17], [19] обучаются либо на каждом инструменте независимо, либо на всей выборке одновременно — в обоих случаях игнорируется латентная кластерная структура ценовых паттернов. Мы явно моделируем эту структуру через совместное обучение эмбеддингов по всем инструментам.

В-третьих, методы обучения представлений для временных рядов [11], [14] тестировались преимущественно на задачах классификации. Их применение к задаче кластеризации рыночных режимов с целью адаптивного прогнозирования доходности остаётся малоизученным.

В-четвёртых, ни одна из рассмотренных работ не реализует полную цепочку: кросс-инструментальные эмбеддинги → кластеризация режимов → кластерно-адаптивный прогноз → портфельная оптимизация → бэктест на данных MOEX. Настоящая работа закрывает этот пробел.

2 Описание общего пайплайна

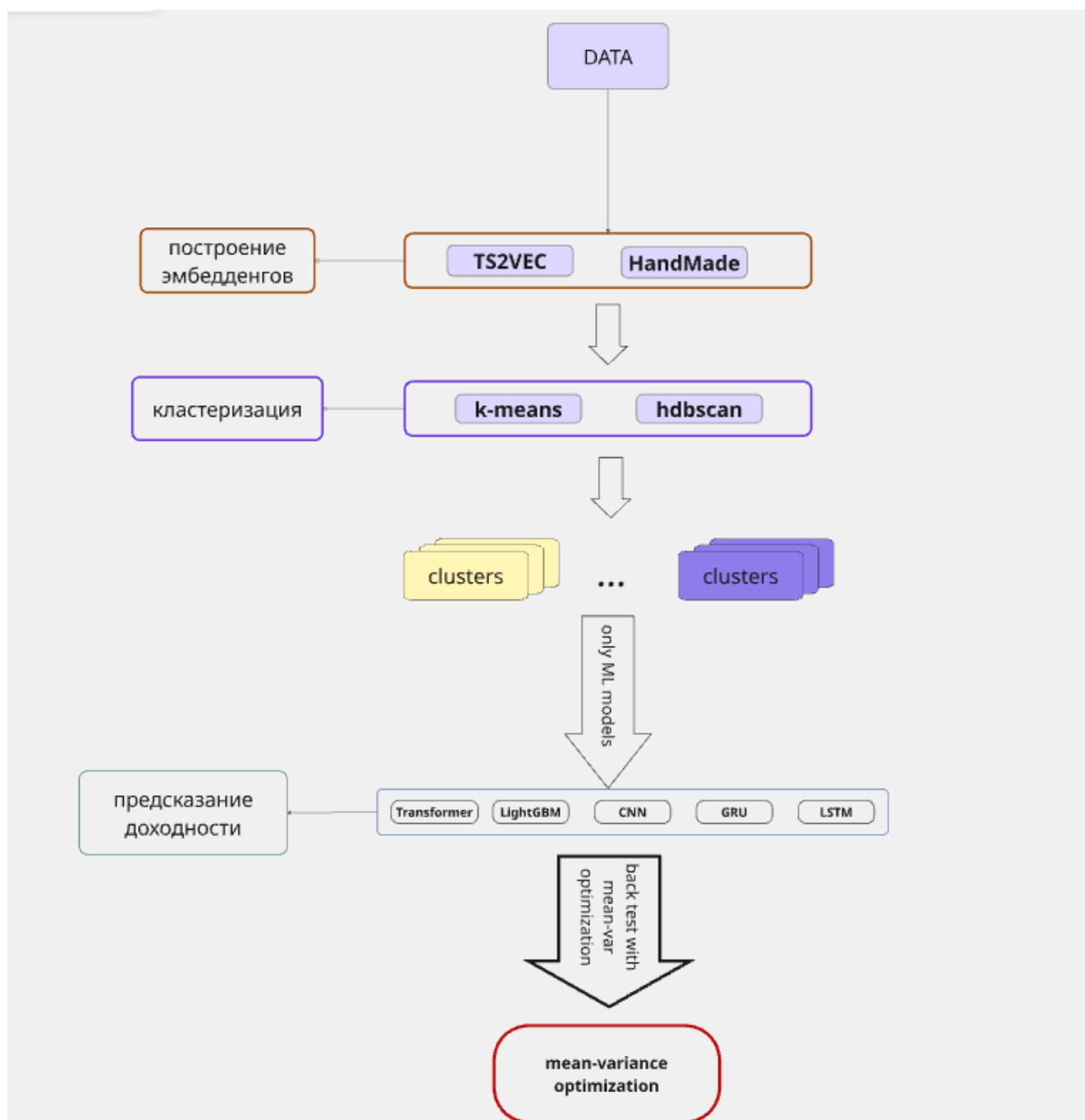


Рис. 2.1. Наш подход

Наша гипотеза состоит из нескольких подходов: построение эмбеддингов, затем кластеризация эмбеддингов, а затем прогнозирование доходностей на каждом кластере. Причём для каждого из этапов нам необходимо искать оптимальное решение. Т. е. нам необходимо проверить, с каким алгоритмом кластеризации наши эмбеддинги работают лучше всего: HDBSCAN [25] или *k*-means [26], сравнить их по метрикам и найти лучший, а также подобрать оптимальное число кластеров, чтобы было достаточно данных для обучения, но и кластеры не сливались в кучу. А далее для каждого кластера нам необходимо перебрать все модели и посмотреть, какая даст наилучший результат. Итогово мы определим лучшее разбиение по кластерам и лучшую модель в каждом кластере.

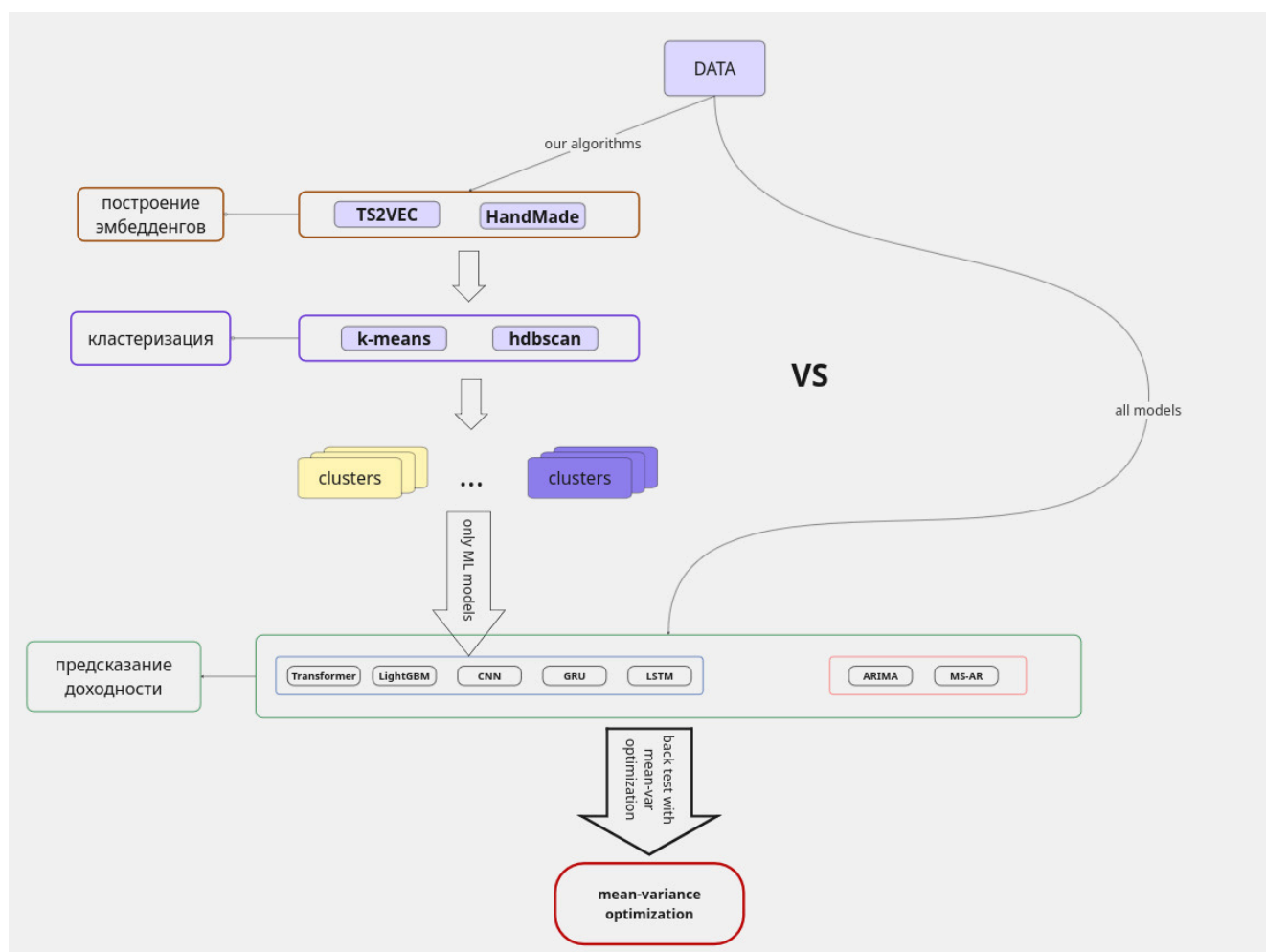


Рис. 2.2. Общая схема пайплайна

Далее необходимо провести детальное сравнение с базовыми моделями, обучив их на всех акциях сразу, и провести бэктестирование.

3 Работа с данными

3.1 Данные

В работе используются дневные данные о свечах всех валидных акций Московской биржи. Свечи базово включают 5 основных параметров помимо timestamp:

open	high	low	close	volume
------	------	-----	-------	--------

где **open** — цена открытия свечи, **high** — максимальная цена свечи, **low** — минимальная цена свечи, **close** — цена закрытия свечи, **volume** — объём торгов.

Эти данные были собраны за период с начала существования акции на бирже вплоть до 2026 года.



Рис. 3.3. Дневные свечи акции SBER

3.2 Предобработка данных и создание признаков

Перед добавлением признаков из данных были удалены пропуски, а также исключены все акции, у которых было менее 252 торговых дней. Таким образом из 260 акций осталось 236.

Далее для каждого временного ряда были добавлены следующие признаки:

sma5	sma20
ema12	ema26
close_sma20	macd
macd_signal	macd_hist
rsi14	bb_pct
bb_bw	atr14
obv	

где признаки вычисляются по формулам:

- $sma5_t = \frac{1}{5} \sum_{i=0}^4 close_{t-i}$ - средняя цена за 5 дней
- $sma20_t = \frac{1}{20} \sum_{i=0}^{19} close_{t-i}$ - средняя цена за 20 дней

- $\text{ema12}_t = \alpha_{12}\text{close}_t + (1 - \alpha_{12})\text{ema12}_{t-1}$, $\alpha_{12} = \frac{2}{12+1}$ - экспоненциальная средняя цена за 12 дней
 - $\text{ema26}_t = \alpha_{26}\text{close}_t + (1 - \alpha_{26})\text{ema26}_{t-1}$, $\alpha_{26} = \frac{2}{26+1}$ - экспоненциальная средняя цена за 26 дней
 - $\text{close_sma20}_t = \frac{\text{close}_t}{\text{sma20}_t}$ - отношение цены к средней цене за 20 дней
 - $\text{macd}_t = \text{ema12}_t - \text{ema26}_t$ - разница между экспоненциальными средними за 12 и 26 дней
 - $\text{macd_signal}_t = \text{ema9}(\text{macd}_t)$ - экспоненциальная скользящая средняя за 9 дней от macd_t
 - $\text{macd_hist}_t = \text{macd}_t - \text{macd_signal}_t$ - разница между macd_t и macd_signal_t
 - $\text{rsi14}_t = 100 - \frac{100}{1+\text{RS}_t}$, $\text{RS}_t = \frac{\text{avgGain14}_t}{\text{avgLoss14}_t}$ - индекс относительной силы за 14 дней
 - $\text{sigma20}_t = \text{std}(\text{close}_{t-19}, \dots, \text{close}_t)$ - стандартное отклонение цены за 20 дней
 - $\text{bb_upper}_t = \text{sma20}_t + 2 \times \text{sigma20}_t$, $\text{bb_lower}_t = \text{sma20}_t - 2 \times \text{sigma20}_t$ - верхняя и нижняя границы полос Боллинджера
 - $\text{bb_pct}_t = \frac{\text{close}_t - \text{bb_lower}_t}{\text{bb_upper}_t - \text{bb_lower}_t}$ - относительное положение цены внутри полос Боллинджера
 - $\text{bb_bw}_t = \frac{\text{bb_upper}_t - \text{bb_lower}_t}{\text{sma20}_t}$ - ширина полос Боллинджера
 - $\text{TR}_t = \max(\text{high}_t - \text{low}_t, \text{abs}(\text{high}_t - \text{close}_{t-1}), \text{abs}(\text{low}_t - \text{close}_{t-1}))$ - истинный диапазон
 - $\text{atr14}_t = \text{ema14}(\text{TR}_t)$ - экспоненциальная средняя истинного диапазона за 14 дней
 - $\text{obv}_t = \text{obv}_{t-1} + \text{sign}(\text{close}_t - \text{close}_{t-1}) \times \text{volume}_t$ — накопленный баланс объёма торгов
- где close_t , high_t , low_t , volume_t — значения цены закрытия, максимума, минимума и объёма в день t соответственно.

Таким образом, всего получается 18 признаков, на которых обучаются модели.

open	high
low	close
volume	sma5
sma20	ema12
ema26	close_sma20
macd	macd_signal
macd_hist	rsi14
bb_pct	bb_bw
atr14	obv

Все данные были разделены на три непересекающихся временных выборки: тренировочную, валидационную и тестовую. Разбиение выполнено по дате, а не случайно, чтобы исключить утечку данных из будущего в прошлое.

Поскольку часть акций появилась на МОЕХ лишь в 2014–2015 годах, тренировочная выборка охватывает наибольший временной промежуток.

Выборка	Период	Доля
----------------	---------------	-------------

Train	2000 — 2022	5/7
Validation	2023 — 2024	1/7
Test	2025 — март 2026	1/7

4 Методы построения эмбеддингов

Кластеризация временных окон возможна только после приведения их к единому векторному формату. Поэтому для каждого окна строится эмбеддинг — числовое представление, сохраняющее информацию о структуре и динамике временного ряда.

Для построения эмбеддингов были использованы следующие методы:

- Handmade Embedder — метод построения эмбеддингов на основе ручных статистических признаков.
- TS2Vec Embedder — метод построения эмбеддингов на основе самообучаемой модели TS2Vec.

4.1 TS2Vec Embedder

TS2Vec [11] — метод построения эмбеддингов временных рядов без учителя, основанный на иерархическом контрастном обучении.

На каждой итерации модель получает окно временного ряда и порождает две аугментации: случайно обрезает начало/конец подпоследовательности и маскирует отдельные временные шаги.

Обе аугментации подаются в один и тот же энкодер, после чего вычисляется контрастная функция потерь: эмбеддинги одного и того же момента времени из разных аугментаций притягиваются, а эмбеддинги разных моментов — отталкиваются.

Энкодер состоит из проекционного слоя в размерность `hidden_dim` и `depth` дилатированных свёрточных блоков (TCN).

После энкодера представления проецируются в пространство размерности `output_dim`.

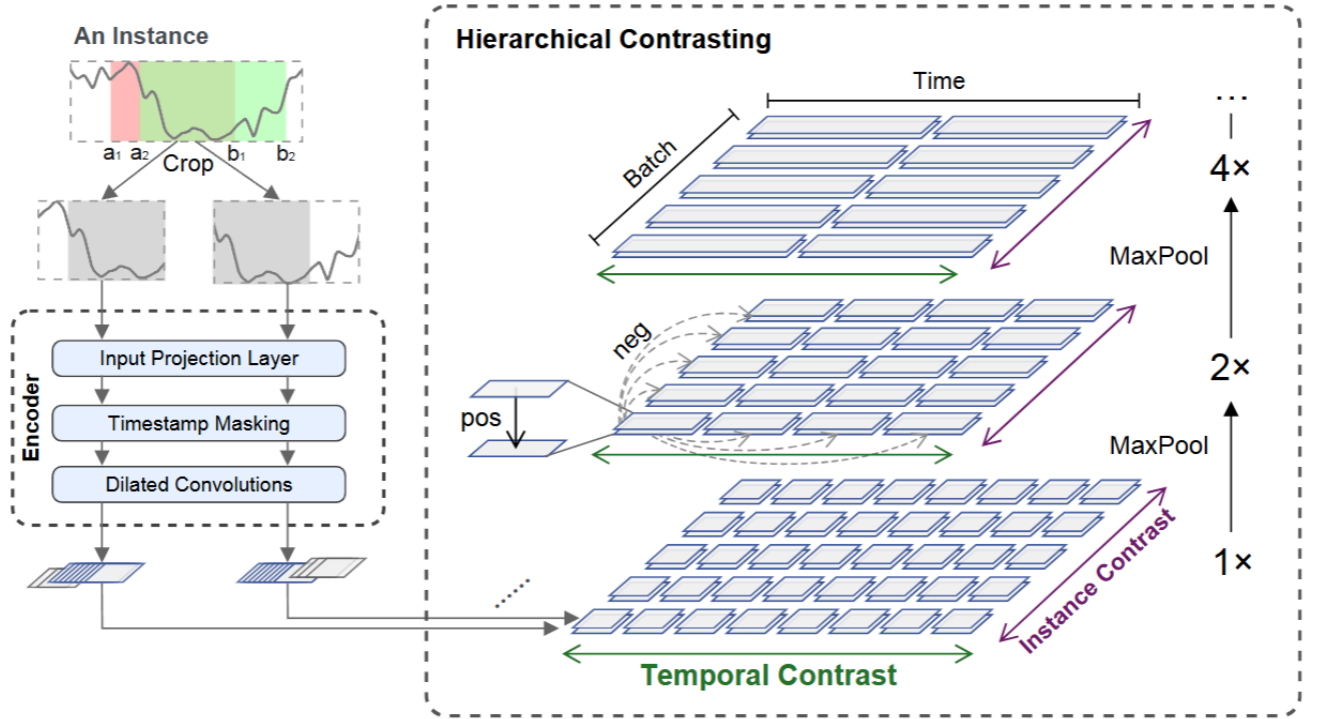


Рис. 4.4. Схема архитектуры TS2Vec

Для построения эмбедингов был выбран $\text{output_dim} = 50$ для окон длиной 60 дней. На обучающей выборке выполнен перебор гиперпараметров энкодера hidden_dim и depth с помощью Optuna [27].

Параметр	Значение
hidden_dim	128
depth	8
output_dim	50

Таблица 4.1. Гиперпараметры TS2Vec после подбора Optuna

4.2 Handmade Embedder

Handmade Embedder — детерминированный метод построения эмбедингов из интерпретируемых признаков окна [12], не требующий обучения нейросети. Для каждого окна длиной $T = 60$ дней вычисляется вектор из 15 признаков, который и является его эмбедингом.

Обозначим z_t — z -нормированное значение close в момент t , v_t — volume, T — длина окна.

Трендовые признаки

- $\text{slope} = \frac{\sum_{t=1}^T (t-\bar{t})(z_t - \bar{z})}{\sum_{t=1}^T (t-\bar{t})^2}$ — линейный наклон ряда
- $\text{net_move} = z_T - z_1$ — суммарное ценовое движение от начала к концу окна
- $\text{frac_above_mean} = \frac{1}{T} \sum_{t=1}^T \mathbb{1}[z_t > \bar{z}]$ — доля баров, на которых цена выше среднего, значение 0.5 соответствует боковому, близкое к 0 или 1 — устойчивому тренду
- $\text{price_range} = \max_t z_t - \min_t z_t$ — размах ценового движения внутри окна

Волатильность

- $\text{daily_vol} = \text{std}(z_t - z_{t-1}, t = 2 \dots T)$ — стандартное отклонение дневных приращений
- $\text{skewness} = \frac{1}{T} \sum_{t=1}^T \left(\frac{z_t - \bar{z}}{\sigma_z} \right)^3$ — асимметрия распределения значений close, положительный скос указывает на редкие резкие подъёмы, отрицательный — на резкие падения
- $\text{kurtosis} = \frac{1}{T} \sum_{t=1}^T \left(\frac{z_t - \bar{z}}{\sigma_z} \right)^4 - 3$ — эксцесс, высокие значения указывают на тяжёлые хвосты и склонность к выбросам

Объёмные признаки

- $\text{price_vol_corr} = \frac{\sum_{t=1}^T \tilde{z}_t \tilde{v}_t}{\sqrt{\sum_{t=1}^T \tilde{z}_t^2 \cdot \sum_{t=1}^T \tilde{v}_t^2 + \varepsilon}}$ — корреляция цены и объёма, где $\tilde{z}_t = z_t - \bar{z}$, $\tilde{v}_t = v_t - \bar{v}$, положительные значения означают, что рост цены подтверждается объёмом
- $\text{vol_activity} = \text{std}(v_t - v_{t-1}, t = 2 \dots T)$ — стандартное отклонение приращений объёма, высокие значения характерны для периодов аномальной торговой активности

Снимок индикаторов в конце окна

- $\text{last_rsi} = \text{RSI}_T$ — значение RSI в последний день окна, отражает текущую перекупленность или перепроданность
- $\text{last_macd_hist} = \text{MACD_hist}_T$ — значение MACD-гистограммы в конце окна, знак указывает на направление импульса
- $\text{last_bb_pct} = \text{BB_pct}_T$ — значение BB%b в конце окна, показывает положение цены относительно полос Боллинджера
- $\text{last_bb_bw} = \text{BB_bw}_T$ — ширина полос Боллинджера в конце окна, высокие значения соответствуют высокой волатильности

Наклон индикаторов внутри окна

- $\text{rsi_slope} = \text{RSI}_T - \text{RSI}_1$ — изменение RSI от начала к концу окна, положительные значения указывают на нарастание импульса
- $\text{macd_slope} = \text{MACD_hist}_T - \text{MACD_hist}_1$ — изменение MACD-гистограммы, показывает, усиливается или затухает текущий импульс

Итоговый вектор эмбединга имеет размерность 15. Все нечисловые значения (NaN, Inf) заменяются нулём.

5 Кластеризация эмбедингов

Кластеризация представляет собой процесс группировки объектов (в нашем случае — эмбедингов) на основе их схожести.

Для кластеризации были использованы следующие методы:

- KMeans — метод кластеризации на основе k -средних.
- HDBSCAN — иерархический плотностной метод кластеризации.

5.1 KMeans

KMeans — метод, который принимает на вход k — количество кластеров и определяет центроиды кластеров.

На каждой итерации алгоритм вычисляет расстояние от каждого объекта до каждого центроида и относит объект к ближайшему кластеру, после чего пересчитывает центроид как среднее арифметическое координат всех объектов кластера. Этот процесс повторяется до тех пор, пока центроиды не стабилизируются.

Подбор оптимального числа кластеров проводился по метрике **silhouette** (коэффициент силуэта) [28].

Пусть $d(i, j)$ — расстояние между объектами i и j в пространстве эмбедингов. Обозначим через $C(i)$ кластер, которому принадлежит объект i .

Внутрикластерная средняя дистанция:

$$a(i) = \frac{1}{|C(i)| - 1} \sum_{j \in C(i), j \neq i} d(i, j) \quad (5.1)$$

Минимальная средняя дистанция до «чужого» кластера:

$$b(i) = \min_{C \neq C(i)} \frac{1}{|C|} \sum_{j \in C} d(i, j) \quad (5.2)$$

Силуэт точки i :

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (5.3)$$

Средний силуэт по выборке из n объектов:

$$\text{Silhouette} = \frac{1}{n} \sum_{i=1}^n s(i) \quad (5.4)$$

Чем ближе значение к 1, тем лучше разделимость кластеров. Значения около 0 указывают на перекрытие кластеров.

Поскольку соседние окна с большим перекрытием с высокой вероятностью принадлежат одному кластеру, окна формировались не подряд с шагом 1 день, а с шагом 5 дней. Это позволяет избежать избыточного дублирования информации и при этом незначительно сокращает общий объём данных.

5.1.1 Визуализация и анализ кластеров

5.1.1.1 Метрики анализа кластеров

Для анализа кластеров были вычислены следующие метрики:

- **mean return** — средняя доходность по кластеру, позволяющая судить о прибыльности соответствующего рыночного режима.
- **std return** — стандартное отклонение доходности по кластеру, характеризующее рискованность режима.

- **sharpe (proxy)** — прокси-коэффициент Шарпа по кластеру, описывающий соотношение доходности и риска.
- **kurtosis** — эксцесс распределения доходности, отражающий «тяжесть» хвостов и склонность к экстремальным значениям.
- **skewness** — асимметрия распределения доходности, показывающая, в какую сторону смещён хвост распределения.

5.1.1.2 TS2Vec

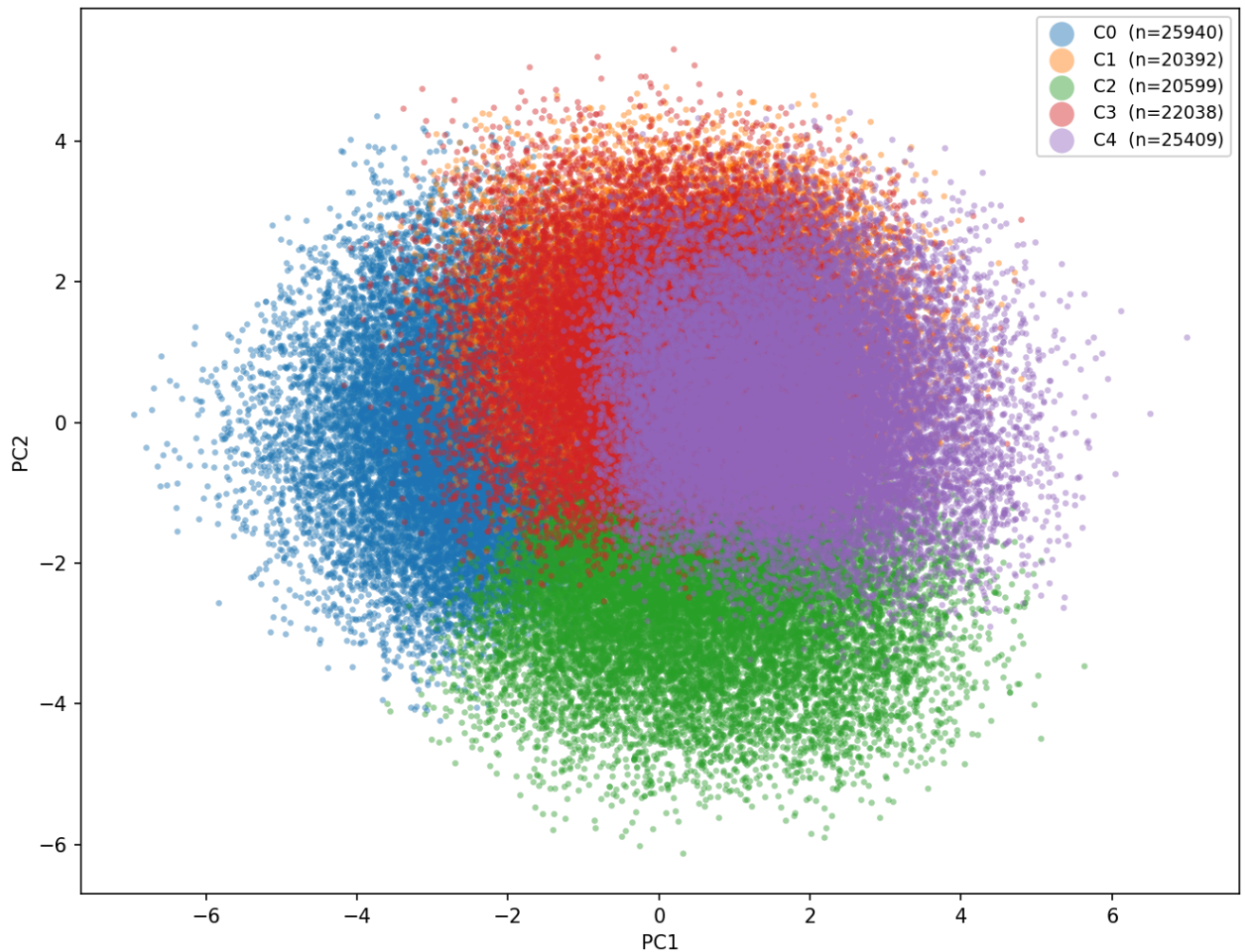


Рис. 5.5. Визуализация кластеров TS2Vec

При использовании TS2Vec для построения эмбедингов наилучший результат получился при числе кластеров $k = 5$ и коэффициенте силуэта $\text{silhouette} = 0.11$.

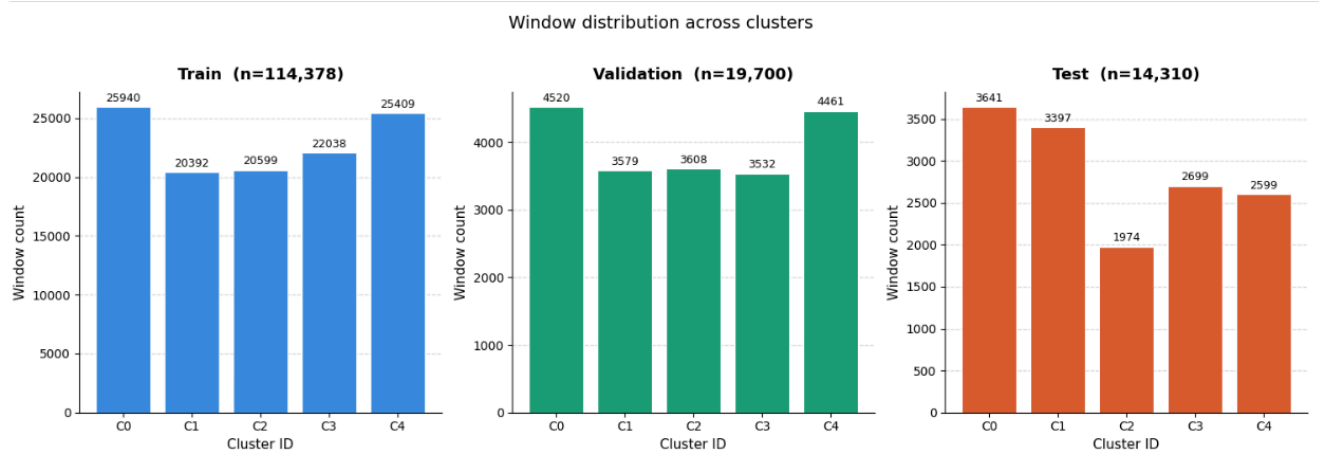


Рис. 5.6. Распределение кластеров TS2Vec

Как видно из графика, кластеры содержат примерно одинаковое количество объектов. Это говорит о том, что кластеризация прошла успешно и не возникло вырожденных кластеров с критически малым числом элементов, на которых было бы невозможно обучить модель. Сбалансированность распределения свидетельствует о том, что алгоритм K-Means нашёл устойчивое разбиение пространства эмбедингов: ни один кластер не доминирует над остальными и не поглощает большую часть обучающей выборки. Это важное свойство с практической точки зрения — каждый кластер представляет собой отдельный режим рынка с достаточным количеством примеров для надёжного обучения специализированных моделей прогнозирования.

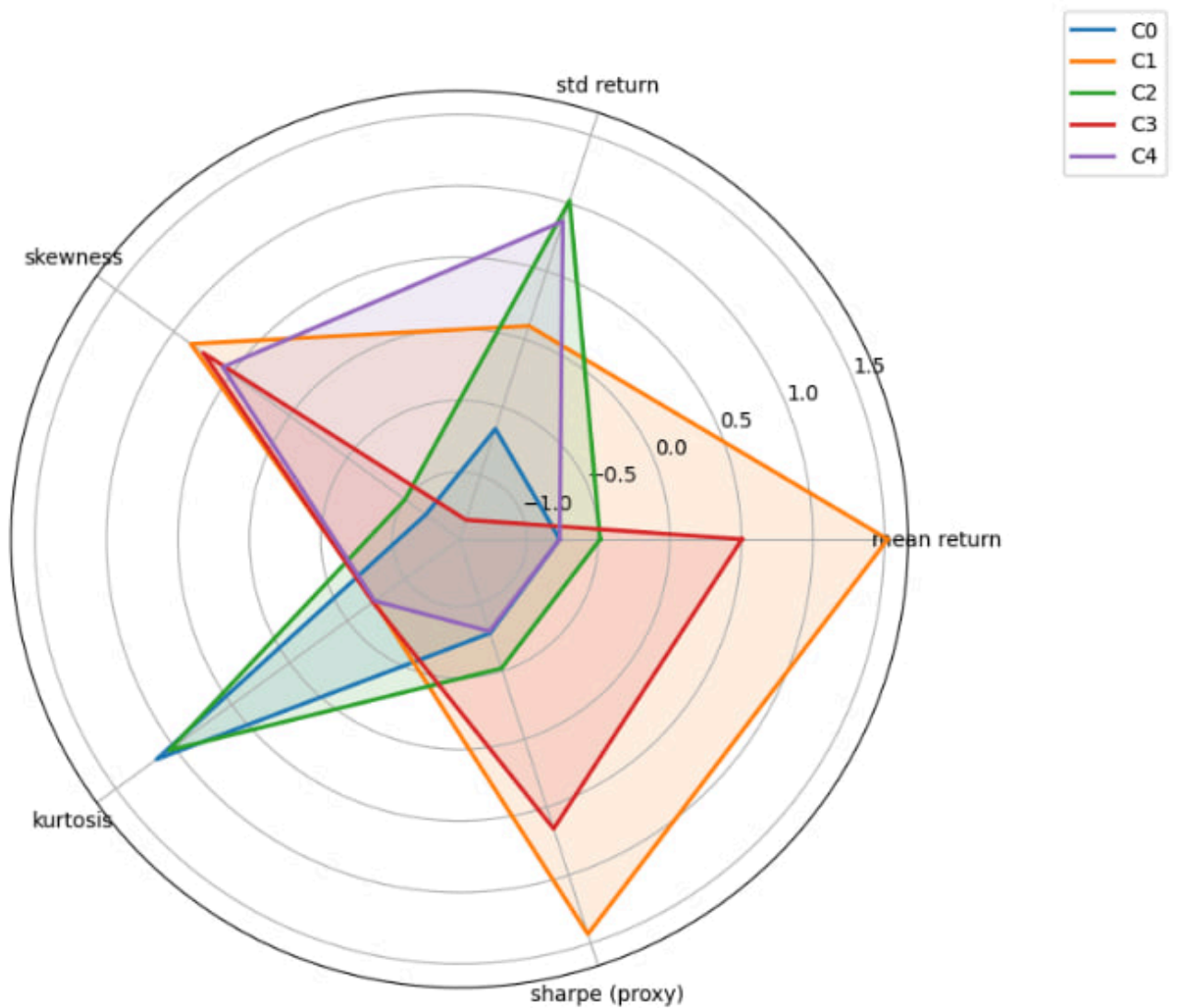


Рис. 5.7. Метрики анализа кластеров TS2Vec

Кластер C0 — нейтральный (боковое движение).

Все метрики данного кластера располагаются вблизи нулевой отметки на радарном графике. Средняя доходность и коэффициент Шарпа близки к нулю, волатильность умеренная, асимметрия и эксцесс не выражены. Такие окна соответствуют периодам рыночной неопределённости, когда цена движется в узком диапазоне без устойчивого тренда. С точки зрения торговой стратегии данный кластер наименее информативен и характеризует «боковик».

Кластер C1 — устойчивый рост (бычий тренд).

Кластер демонстрирует наибольшую среднюю доходность и наиболее высокий прокси-коэффициент Шарпа среди всех кластеров. Волатильность при этом остаётся умеренной, что обеспечивает благоприятное соотношение доходности к риску. Положительная асимметрия распределения указывает на то, что редкие крупные прибыльные движения присутствуют чаще, чем убыточные выбросы. Данный кластер соответствует устойчивым бычьим периодам рынка, на которых длинные позиции наиболее эффективны.

Кластер C2 — высокая волатильность (нестабильный рынок).

На данном кластере наблюдается высокое стандартное отклонение доходности и высокий эксцесс распределения, тогда как средняя доходность близка к нулю. Это свидетельствует о наличии резких ценовых движений в обоих направлениях: как сильных ростов, так и глубоких падений.

Кластер C3 — снижение (медвежий тренд).

Данный кластер характеризуется отрицательной средней доходностью и отрицательным значением прокси-коэффициента Шарпа, что однозначно указывает на медвежий режим рынка.

Кластер C4 — аномальный (экстремальные движения).

Кластер характеризуется высокой положительной асимметрией при умеренных значениях остальных метрик: большинство дней спокойные, однако редкие резкие скачки вверх сильно смещают распределение вправо. Такой профиль типичен для периодов резкого отскока после коррекции или реакции на неожиданно позитивные новости.

5.1.1.3 Handmade

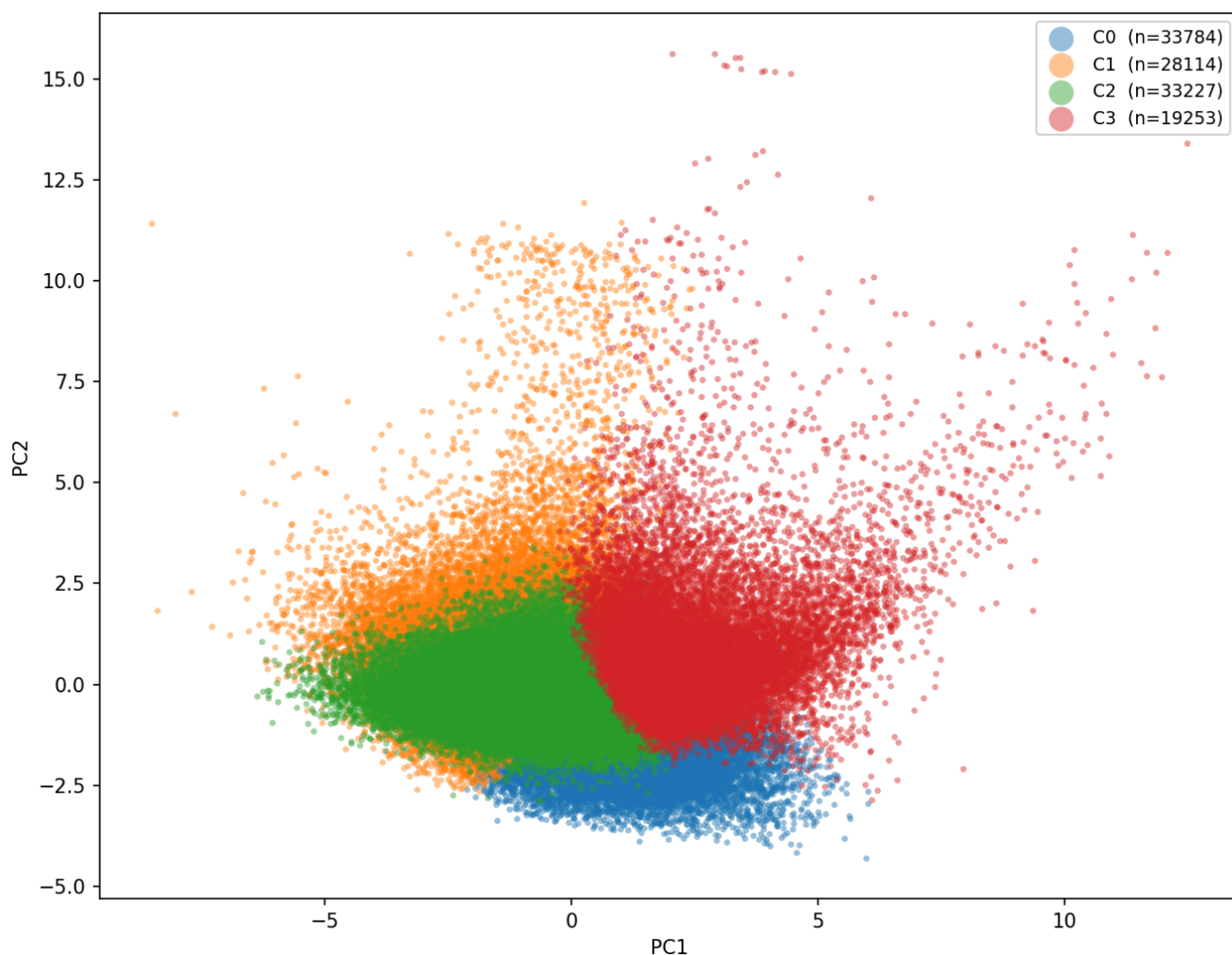


Рис. 5.8. Визуализация кластеров Handmade

При использовании ручных (handmade) признаков для построения эмбедингов наилучший результат получился при числе кластеров $k = 4$ и коэффициенте силуэта silhouette = 0.16.

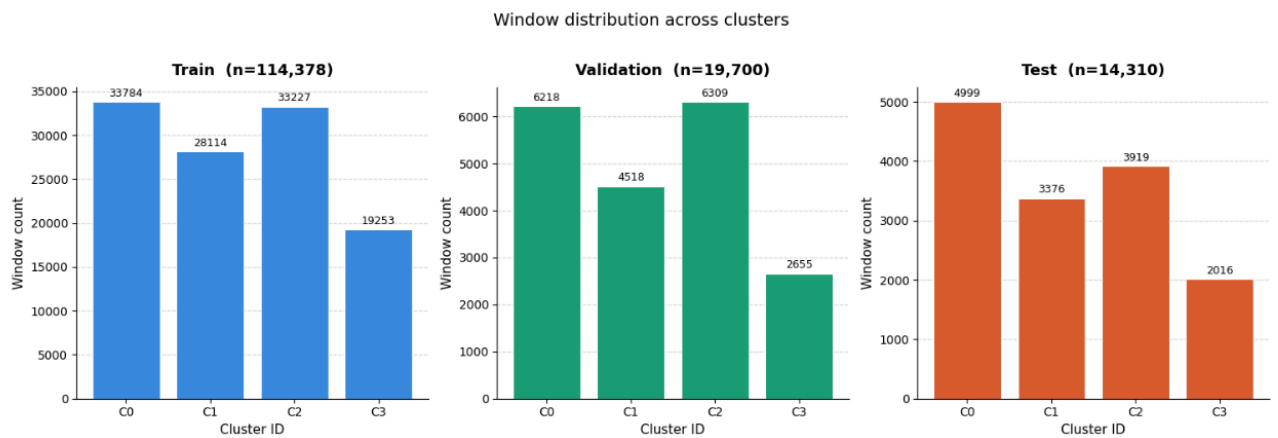


Рис. 5.9. Распределение кластеров Handmade

Распределение окон по кластерам в целом умеренно сбалансированно: кластеры C0, C1 и C2 содержат сопоставимое число объектов (28–34 тысячи на обучающей выборке), тогда как кластер C3 заметно меньше — около 19 тысяч окон. Аналогичная пропорция сохраняется на валидационной и тестовой выборках, что подтверждает устойчивость разбиения. Относительно меньший размер C3 обусловлен редкостью соответствующего рыночного режима, однако его объём остаётся достаточным для обучения отдельной модели.

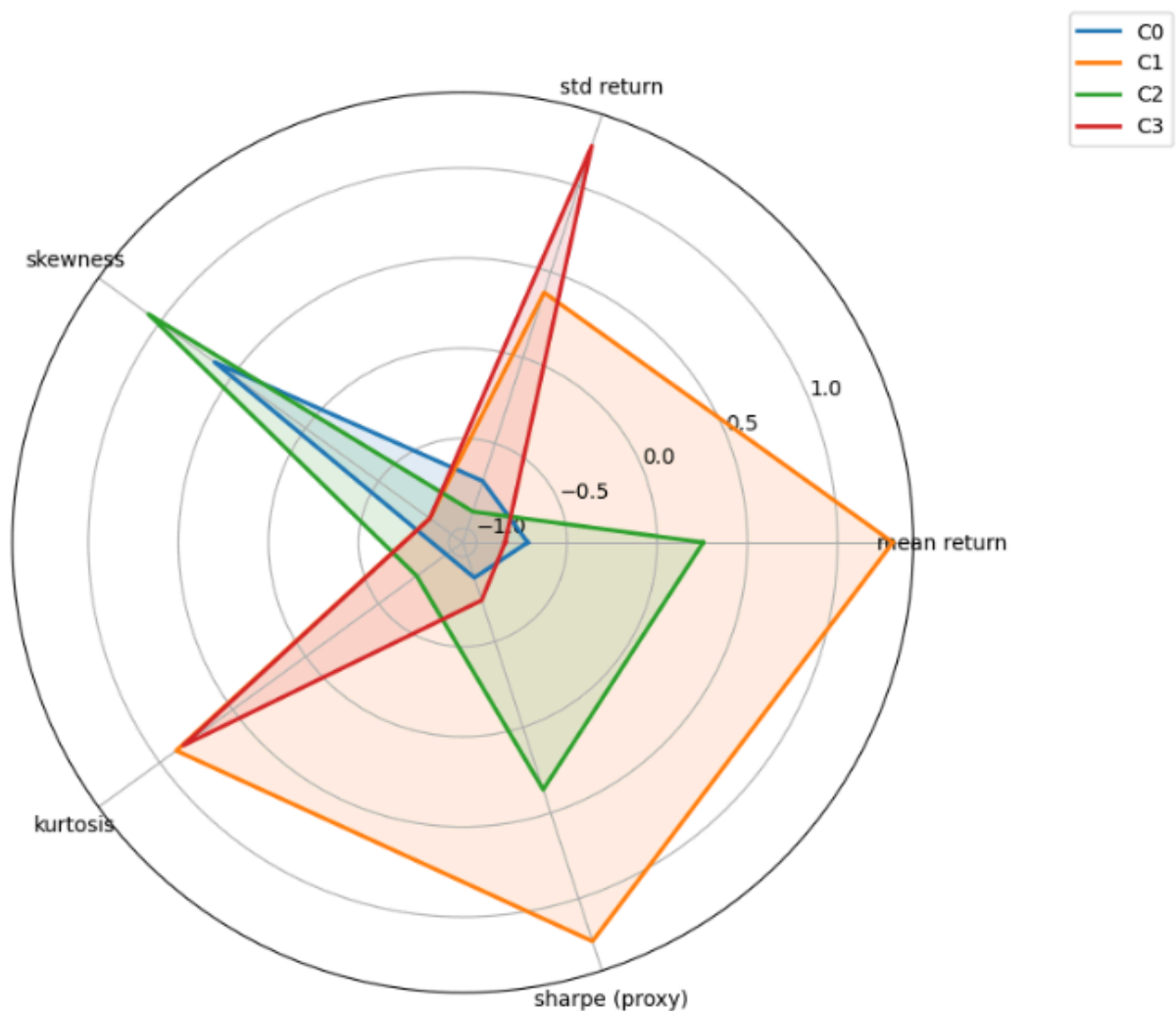


Рис. 5.10. Метрики анализа кластеров Handmade

Кластер C0 — нейтральный (боковое движение).

На радарном графике все метрики данного кластера сосредоточены вблизи нулевой отметки и не имеют выраженных пиков. Средняя доходность, стандартное отклонение, коэффициент Шарпа, асимметрия и эксцесс — все принимают умеренные значения. Данный кластер соответствует периодам низкой активности рынка без выраженного направления движения.

Кластер C1 — устойчивый рост (бычий тренд).

Кластер демонстрирует наибольшую среднюю доходность и наибольший прокси-коэффициент Шарпа среди всех четырёх кластеров, при умеренной волатильности. Сочетание высокой средней доходности и относительно небольшого стандартного отклонения формирует максимально благоприятный профиль риск/доходность. Данный кластер соответствует устойчивым периодам роста, на которых стратегия удержания длинной позиции наиболее эффективна.

Кластер C2 — нисходящий рынок с отрицательной асимметрией.

Отличительная черта кластера — выраженный отрицательный эксцесс (kurtosis) и отрицательная асимметрия (skewness), при этом средняя доходность близка к нулю или слабо отрицательна. Отрицательный эксцесс указывает на «плосковершинное» распределение с укороченными хвостами — доходности менее склонны к экстремальным значениям, чем нормальное распределение. Отрицательный скос свидетельствует о том, что редкие крупные убытки всё же присутствуют, тогда как прибыльные движения более равномерны. Подобный профиль характерен для вялотекущего снижения без резких скачков.

Кластер C3 — высокая волатильность с падением (кризисный режим).

Кластер отличается наибольшей волатильностью, отрицательной средней доходностью и худшим коэффициентом Шарпа. Рынок в этом режиме резко падает, но с периодическими сильными отскоками, что делает прогнозирование особенно затруднительным. Такие периоды соответствуют рыночным кризисам и паническим распродажам.

5.1.1.4 Сравнение TS2Vec и Handmade

Сопоставляя результаты кластеризации на основе handmade-признаков и TS2Vec-эмбеддингов, можно отметить, что оба подхода выделяют схожие рыночные режимы: нейтральный, бычий, медвежий и кризисный.

Из полученных результатов можно сделать вывод, что кластеры действительно отражают различные режимы рынка, а гипотеза о целесообразности использования разных моделей для разных режимов подтверждается.

Примечательно, что при использовании handmade-признаков коэффициент силуэта оказался выше, что свидетельствует о более чётком разделении кластеров.

Это объясняется тем, что статистики, вычисленные по данным окна, непосредственно описывают характер ценового движения и потому хорошо представляют различные режимы рынка.

TS2Vec, в свою очередь, строит более богатые контекстные представления, однако его эмбединги несут информацию о тонких временных зависимостях, которые K-Means труднее разделяет.

5.2 HDBSCAN

HDBSCAN — плотностной иерархический метод кластеризации, принимающий на вход минимальный размер кластера. Перед кластеризацией данные масштабируются.

Метод обнаруживает плотные области и объединяет их в кластеры, не требуя явного задания их числа. С одной стороны, это позволяет избежать перебора k , с другой — на финансовых данных алгоритм, как правило, выделяет один-два крупных кластера и большое число мелких, непригодных для обучения модели.

Крупные кластеры в данном случае несут мало специфической информации и фактически разделяют окна лишь по уровню доходности, тогда как мелкие кластеры улавливают индивидуальную структуру окон, но содержат порядка 1000 объектов — слишком мало для надёжного обучения.

6 Модели прогнозирования доходности

В каждом из экспериментов — с разными видами кластеризации и без — используется набор моделей, описанный ниже.

Все модели, кроме ARIMA, решают задачу регрессии: по скользящему окну длины T предсказать логарифмическую доходность следующего шага:

$$\ln \frac{p_{t+1}}{p_t} \quad (6.5)$$

где p_t — цена закрытия последнего дня окна. ARIMA работает иначе: принимает на вход лог-ценовой ряд $\ln(p_t)$ и предсказывает следующее значение $\ln(p_{t+1})$, из которого логарифмическая доходность восстанавливается как разность $\ln(p_{t+1}) - \ln(p_t)$.

6.1 Подбор гиперпараметров

Для каждой модели реализован автоматический поиск гиперпараметров с использованием фреймворка Optuna (кроме ARIMA), который оптимизирует IC – Information Coefficient:

$$IC = \frac{\sum_i (y_i^{\text{true}} - \bar{y}^{\text{true}})(y_i^{\text{pred}} - \bar{y}^{\text{pred}})}{\sqrt{\sum_i (y_i^{\text{true}} - \bar{y}^{\text{true}})^2 \cdot \sum_i (y_i^{\text{pred}} - \bar{y}^{\text{pred}})^2}}. \quad (6.6)$$

В качестве метрики оптимизации IC был выбран по двум причинам:

1. Модели, оптимизированные по IC, обычно превосходят модели, оптимизированные по MSE, на задачах прогнозирования финансовых рядов
2. При низком IC классический Марковиц дает огромные ошибки в весах [23]

6.2 ARIMA

6.2.1 Архитектура

Модель $ARIMA(p, d, q)$ [8] описывает временной ряд как стационарный процесс после d -кратного дифференцирования. AR-компонент моделирует зависимость от p лагов ряда, MA-компонент — от q лагов ошибки предсказания:

$$\varphi(B)(1 - B)^d y_t = \theta(B)\varepsilon_t, \quad (6.7)$$

где B — оператор сдвига, $\varphi(B) = 1 - \varphi_1 B - \dots - \varphi_p B^p$, $\theta(B) = 1 + \theta_1 B + \dots + \theta_q B^q$, $\varepsilon_t \sim WN(0, \sigma^2)$. Параметры оцениваются методом максимального правдоподобия. ARIMA принимает на вход одномерный лог-ценовой ряд $\ln(p_t)$ для достижения стационарности и предсказывает логарифм следующего члена ряда $\ln(p_{t+1})$.

6.3 Реализация

Поскольку модель применяется лишь для прогона на всех акциях (без кластеризации), то для каждого тикера строится и оптимизируется своя модель, а затем каждая метрика усредняется по всем тикерам.

Для автоматического и честного выбора тройки (p, d, q) используется алгоритм **Auto ARIMA** [29], который итеративно перебирает комбинации параметров в заданных диапазонах, отбирая модель с минимальным значением информационного критерия AIC:

$$AIC = -2 \ln \hat{L} + 2k, \quad (6.8)$$

где $\hat{L}$ — максимум функции правдоподобия модели, k — число оцениваемых параметров.

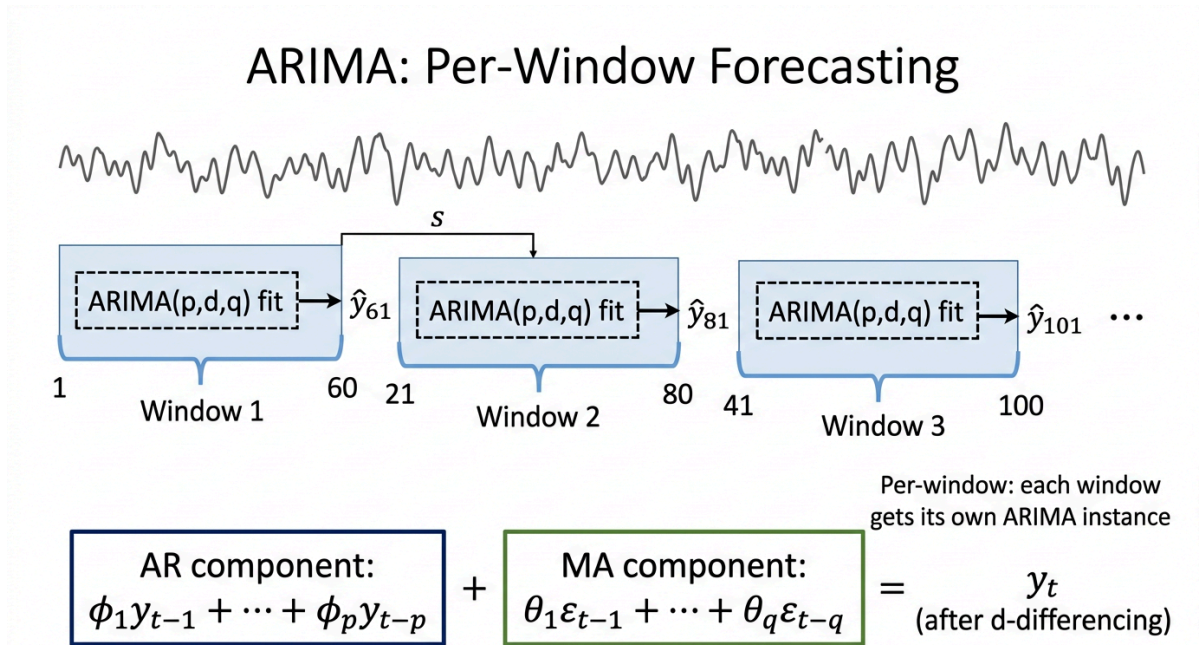


Рис. 6.11. Схема ARIMA в режиме per-window: каждое скользящее окно даёт независимый одношаговый прогноз.

Мы выбрали ARIMA в качестве бейзлайна, потому что ARIMA совершенно не требует GPU, что облегчает её тестирование в локальной среде, а также хорошо улавливает короткосрочные доходности. Модель интерпретируема и корректно улавливает краткосрочную

автокорреляцию доходностей. Однако она линейна, а значит не улавливает нелинейные зависимости, а финансовые ряды обычно обладают нелинейными свойствами, что делает ARIMA менее конкурентоспособной.

6.4 MR-AR

6.4.1 Архитектура

MS-AR (Markov Switching AutoRegression) [9] — обобщение классической AR-модели, в которой параметры процесса переключаются между K скрытыми режимами. Режим $s_t \in \{1, \dots, K\}$ — скрытая дискретная случайная величина, эволюционирующая по однородной цепи Маркова первого порядка с матрицей переходных вероятностей Π , где $\pi_{ij} = P(s_t = j \mid s_{t-1} = i)$.

В каждом режиме k динамика ряда описывается AR(p)-процессом:

$$y_t = c_k + \sum_{l=1}^p \varphi_{k,l} y_{t-l} + \varepsilon_t^{(k)}, \quad \varepsilon_t^{(k)} \sim \mathcal{N}(0, \sigma_k^2), \quad (6.9)$$

где c_k — режим-специфичная константа, $\varphi_{k,1}, \dots, \varphi_{k,p}$ — коэффициенты авторегрессии режима k , σ_k^2 — дисперсия шума (при `switching_variance = True` различается по режимам).

Параметры $\Theta = \{c_k, \varphi_{k,l}, \sigma_k^2, \Pi\}$ оцениваются алгоритмом ЕМ. Е-шаг вычисляет апостериорные вероятности режимов посредством фильтра и сглаживателя Гамильтона [30]. М-шаг обновляет параметры методом взвешенного МНК/MLE.

Одношаговый прогноз строится как взвешенная смесь предсказаний по всем режимам:

$$\hat{y}_{t+1} = \sum_{k=1}^K P(s_t = k \mid y_{1:t}) \cdot \left(c_k + \sum_{l=1}^p \varphi_{k,l} y_{t+1-l} \right), \quad (6.10)$$

где $P(s_t = k \mid y_{1:t})$ — сглаженные вероятности последнего шага (`smoothed_marginal_probabilities`).

6.4.2 Реализация

Модель работает в режиме «per-window»: для каждого тестового окна длины T выполняется независимая подгонка MS-AR на нормализованном ряде `close` данного окна (первый признак входного тензора). Прогноз $\hat{y}_{t+1}$ вычисляется вручную по формуле выше: сглаженные вероятности режимов последнего шага объединяются с соответствующими константами и AR-коэффициентами из `res.params`.

Поскольку каждое окно требует отдельной ЕМ-процедуры, Optuna не применяется — гиперпараметры фиксируются вручную по нескольким пробным прогонам. Базовая конфигурация: $K = 2$ режима, порядок AR $p = 1$, `switching_variance = True`.

MR-AR: Per-Window Markov Switching Forecasting

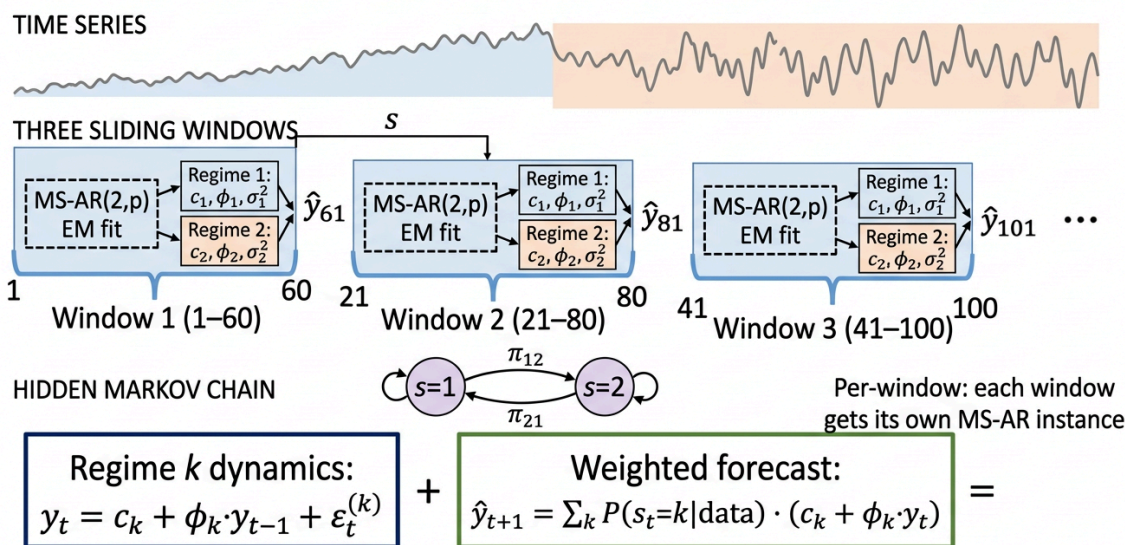


Рис. 6.12. Схема MR-AR в режиме per-window: скользящее окно → ЕМ-оценка параметров двух режимов → взвешенный одношаговый прогноз.

Мы включили MR-AR как интерпретируемое расширение ARIMA, явно учитывающее смену рыночных режимов: периоды тренда и высокой волатильности естественным образом отображаются на разные скрытые состояния. Параметры каждого режима (c_k, ϕ_k, σ_k) и вероятности переключения Π имеют содержательный финансовый смысл, что отличает модель от «чёрных ящиков». Ряд работ показывает, что MS-модели лучше ARIMA описывают структурные сдвиги в финансовых рядах [15].

Ключевое ограничение — высокая вычислительная стоимость: каждое тестовое окно требует отдельного прохода ЕМ, что делает MR-AR значительно медленнее ARIMA. При малом числе наблюдений ($T = 60$) ЕМ может сходиться к вырожденному решению с вырожденными вероятностями режимов.

Гиперпараметры.

Параметр	Описание	Значения	Тип
k_regimes	Число скрытых режимов K	{2, 3}	int
order	Порядок AR p в каждом режиме	{1, 2, 3}	int
switching_variance	Различные σ_k^2 по режимам	{True, False}	bool

Таблица 6.2. Гиперпараметры MR-AR. При $K = 2$ модель выделяет два режима (тренд / флэт). switching_variance = True позволяет режимам различаться по волатильности.

6.5 LightGBM

6.5.1 Архитектура

В основе LightGBM [10] лежит последовательное наращивание ансамбля: на каждом шаге m новое дерево f_m подгоняется к псевдоостаткам,

$$\hat{y}^{(m)} = \hat{y}^{(m-1)} + \eta \cdot f_m(x), \quad (6.11)$$

где η — шаг сжатия. После M итераций итоговый прогноз есть $\hat{y} = \sum_{m=1}^M \eta \cdot f_m(x)$.

От классического GBDT LightGBM отличается двумя инженерными решениями. Leaf-wise рост — вместо того чтобы разбивать все листья одного уровня, алгоритм каждый раз выбирает единственный лист с наибольшим снижением функции потерь. Деревья получаются асимметричными и более глубокими. Чтобы сдерживать переобучение, ёмкость регулируется параметром `num_leaves`, а не глубиной. **Gradient-based One-Side Sampling (GOSS)** — наблюдения с малыми градиентами случайно прореживаются, «трудные» же примеры сохраняются целиком. За счёт этого каждая итерация обрабатывает заметно меньше данных без ощутимых потерь в точности.

6.5.2 Реализация

Поскольку LightGBM работает с табличными данными, трёхмерный тензор (N, T, F) предварительно разворачивается в матрицу $(N, T \cdot F)$: каждая пара (временной шаг, признак) становится отдельным столбцом. Порядок следования шагов при этом сохраняется в индексах столбцов, но сама модель не имеет понятия о последовательности — соседние временные шаги для неё ничем не отличаются от произвольных числовых фич. Ранняя остановка ведётся по валидационному MSE.

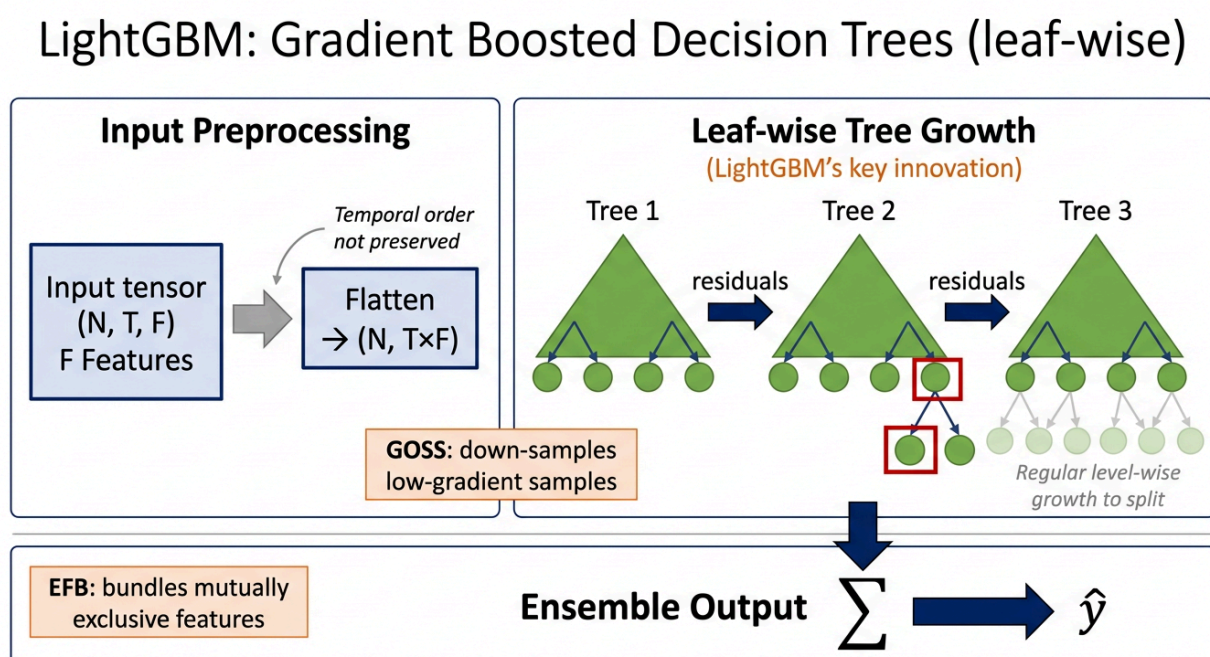


Рис. 6.13. Архитектура LightGBM: flatten временного ряда, leaf-wise рост деревьев, суммирование ансамбля.

LightGBM привлекателен устойчивостью к выбросам, способностью улавливать нелинейные взаимодействия признаков и высокой скоростью обучения на CPU даже при больших объёмах данных. На ряде задач прогнозирования финансовых рядов бустинг на деревьях показывает результаты, сопоставимые с глубокими нейронными сетями [17].

Вместе с тем разворачивание временной оси в плоский вектор приводит к потере информации о порядке: перестановка шагов $x_1, \dots, x_T$ и $x_T, \dots, x_1$ даст идентичный набор признаков без явного позиционного кодирования. Поэтому там, где динамика ряда опре-

деляется именно последовательностью событий, а не их набором, LightGBM уступает рекуррентным архитектурам.

Подбор гиперпараметров (50 проб Optuna).

Параметр	Описание	Диапазон	Тип
n_estimators	Число деревьев в ансамбле	200–1200, шаг 100	int
learning_rate	Шаг сжатия η	0.005–0.2 (log)	float
num_leaves	Макс. листьев на дерево (leaf-wise)	15–255, шаг 16	int
max_depth	Макс. глубина (–1 = без ограничения)	–1–14	int
min_child_samples	Мин. образцов в листе	5–80, шаг 5	int
subsample	Доля строк при построении дерева	0.5–1.0	float
colsample_bytree	Доля признаков при построении дерева	0.5–1.0	float
reg_alpha	L_1 -регуляризация весов листьев	0.0–1.0	float
reg_lambda	L_2 -регуляризация весов листьев	0.0–2.0	float

Таблица 6.3. Пространство поиска LightGBM. num_leaves — главный регулятор ёмкости при leaf-wise росте.

6.6 LSTM

6.6.1 Архитектура

LSTM [4] — рекуррентная архитектура, преодолевающая проблему затухающего градиента классических RNN. Ключевое отличие от RNN — ячейка памяти C_t , отдельная от скрытого состояния h_t и управляемая тремя гейтами:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad (6.12)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad \tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C), \quad (6.13)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \quad (6.14)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(C_t), \quad (6.15)$$

где σ — сигмоида, $\odot$ — поэлементное произведение. Гейт забывания f_t решает, какую долю предыдущей ячейки памяти сохранить. Гейт входа i_t контролирует запись нового кандидата $\tilde{C}_t$. Гейт выхода o_t определяет, что передать в скрытое состояние.

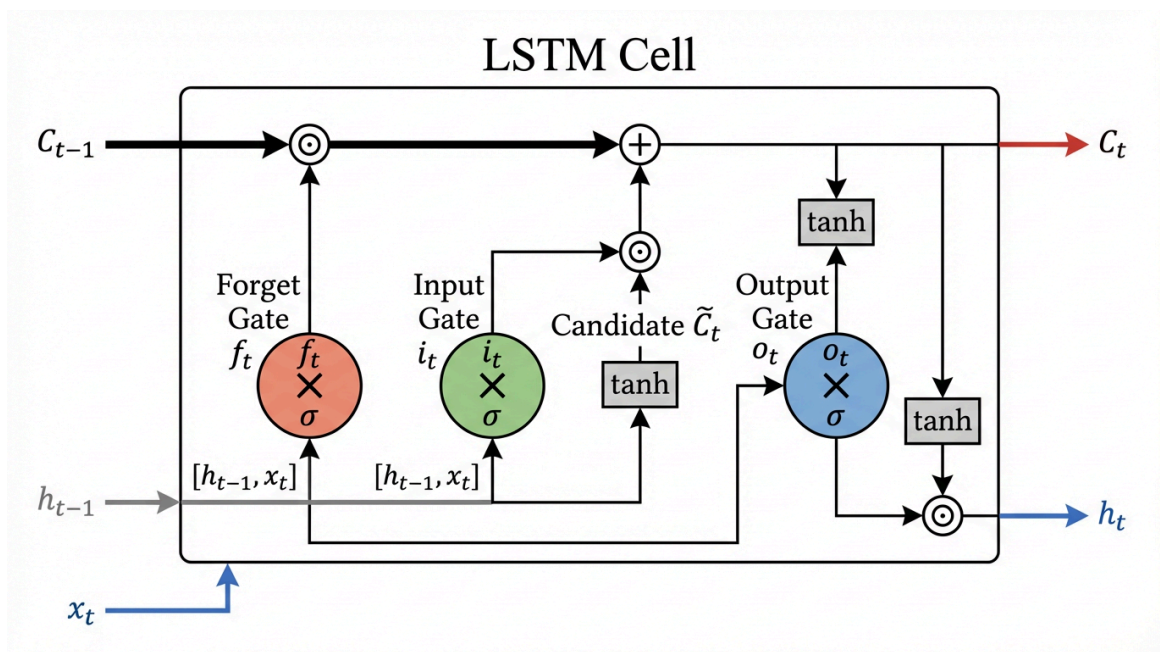


Рис. 6.14. Структура LSTM-ячейки: ячейка памяти C_t (горизонтальный поток) защищена тремя гейтами f_t , i_t , o_t .

6.6.2 Реализация

Стек из `num_layers` LSTM-слоёв с размером скрытого состояния `hidden_size`. Dropout применяется между слоями при `num_layers > 1`. Скрытое состояние последнего шага h_T подаётся в линейный слой $h_T \rightarrow \hat{y}$. Обучение — Adam, потеря — MSE, ранняя остановка по валидационной потере.

Мы выбрали LSTM, потому что гейтовый механизм позволяет выборочно «запоминать» медленно меняющиеся тренды и «забывать» устаревшую информацию — важное свойство для финансовых рядов со сменой режимов. Работа [16] показала, что LSTM на дневных данных S&P 500 статистически значимо превосходит случайный лес по Directional Accuracy, а в [18] LSTM устойчиво превосходит ARIMA по MAE и RMSE.

Несмотря на это, LSTM имеет существенные ограничения: последовательная обработка не параллелизуется, при длинных окнах информация начала может «вымываться» даже с гейтами, а высокое число параметров повышает риск переобучения.

Подбор гиперпараметров (50 проб Optuna).

Параметр	Описание	Диапазон	Тип
<code>hidden_size</code>	Размер скрытого состояния h_t	32–256, шаг 32	int
<code>num_layers</code>	Число стековых LSTM-слоёв	1–3	int
<code>dropout</code>	Dropout между слоями	0.0–0.4	float
<code>lr</code>	Скорость обучения Adam	1e-4–5e-3 (log)	float
<code>batch_size</code>	Размер батча	{32, 64, 128}	cat
<code>epochs</code>	Максимум эпох	20–80, шаг 10	int
<code>patience</code>	Порог ранней остановки	5–15	int

Таблица 6.4. Пространство поиска LSTM (50 проб).

6.7 GRU

6.7.1 Архитектура

Управляемый рекуррентный блок (Gated Recurrent Unit, GRU) [5] упрощает LSTM, объединяя гейт забывания и гейт входа в единый **гейт обновления** z_t и вводя **гейт сброса** r_t :

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r), \quad (6.16)$$

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z), \quad (6.17)$$

$$\tilde{h}_t = \tanh(W[h_t \odot h_{t-1}, x_t] + b), \quad (6.18)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \quad (6.19)$$

Гейт сброса r_t определяет, сколько предыдущего состояния учитывать при формировании кандидата. Гейт обновления z_t задаёт баланс между старым и новым. Ячейка памяти C_t отсутствует — скрытое состояние h_t выполняет обе роли.

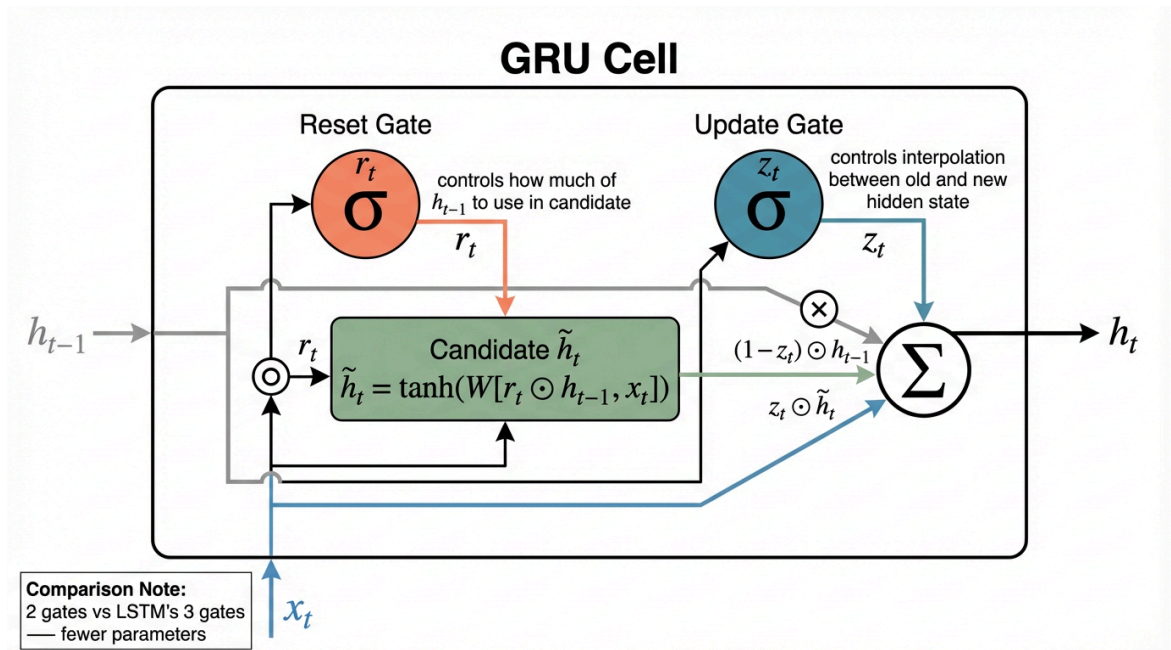


Рис. 6.15. Структура GRU-ячейки: два гейта вместо трёх, нет отдельной ячейки памяти — меньше параметров, чем у LSTM.

6.7.2 Реализация

Архитектура идентична LSTM: стек из `num_layers` GRU-слоёв, `dropout` между ними, скрытое состояние последнего шага h_T подаётся в линейный слой. Обучение — Adam, потеря — MSE, ранняя остановка по валидационной потере.

Мы выбрали GRU как более эффективную альтернативу LSTM: 25% меньше параметров при аналогичном качестве на коротких последовательностях ($T \leq 100$). Исследование [18] показало схожую точность GRU и LSTM на финансовых данных, однако GRU сходиллся быстрее. На датасетах среднего размера GRU нередко является более практичным выбором [17].

Те же структурные ограничения последовательного вычисления, что и у LSTM. При $T = 60$ отсутствие явной ячейки памяти не является критичным.

Подбор гиперпараметров (50 проб Optuna). Пространство поиска идентично LSTM — меньший объём модели при том же бюджете проб позволяет GRU быстрее находить оптимум.

Параметр	Описание	Диапазон	Тип
hidden_size	Размер скрытого состояния h_t	32–256, шаг 32	int
num_layers	Число стековых GRU-слоёв	1–3	int
dropout	Dropout между слоями	0.0–0.4	float
lr	Скорость обучения Adam	1e-4–5e-3 (log)	float
batch_size	Размер батча	{32, 64, 128}	cat
epochs	Максимум эпох	20–80, шаг 10	int
patience	Порог ранней остановки	5–15	int

Таблица 6.5. Пространство поиска GRU (50 проб).

6.8 CNN

6.8.1 Архитектура

Одномерная свёрточная нейронная сеть применяет ядро размера k вдоль временной оси, обнаруживая локальные паттерны независимо от их позиции в окне. Операция свёртки для i -й позиции и j -го фильтра:

$$(X * W)_{i,j} = \sum_{c=1}^F \sum_{\tau=0}^{k-1} X_{i+\tau,c} \odot W_{\tau,c,j} + b_j, \quad (6.20)$$

где $W \in \mathbb{R}^{k \times F \times C}$ — тензор весов, C — число фильтров, k — размер ядра. Дополнение `padding = k // 2` сохраняет длину временной оси.

6.8.2 Реализация

Тензор (batch, T, F) транспонируется в (batch, F, T) для PyTorch Conv1d. Проходит num_layers свёрточных блоков.

AdaptiveAvgPool1d(1) агрегирует временную ось и линейный слой формирует прогноз.

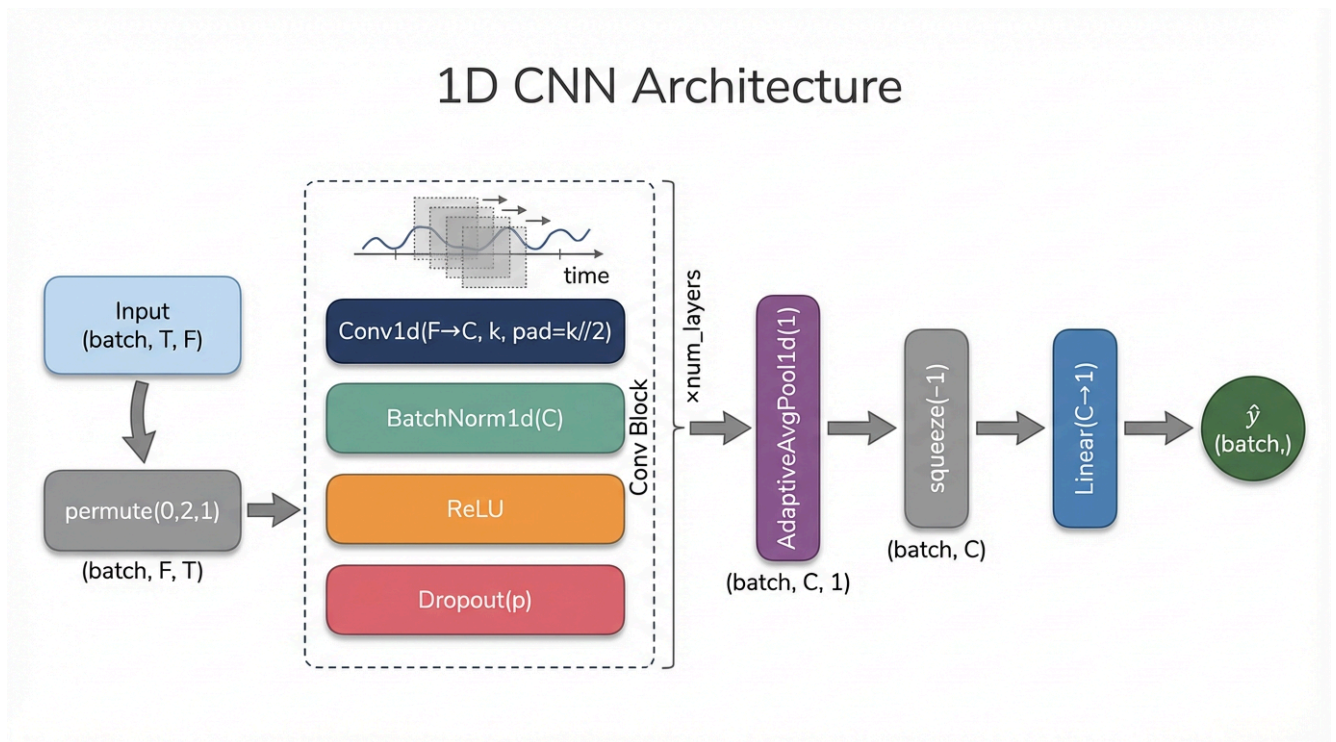


Рис. 6.16. Архитектура 1D CNN: транспонирование, блоки Conv1d→BN→ReLU→Dropout, глобальный Average Pooling, линейный выход.

Мы выбрали CNN, потому что свёртки с различными `kernel_size` улавливают локальные паттерны разных временных масштабов ($k = 3 \approx$ неделя, $k = 9 \approx$ полмесяца), а вычисления полностью параллелизованы — CNN обучается значительно быстрее LSTM/GRU. `BatchNorm` стабилизирует обучение на нестационарных данных [17].

Рецептивное поле ограничено $L = \text{num_layers} \times k$, поэтому длинные зависимости CNN улавливает хуже рекуррентных моделей. `AdaptiveAvgPool1d` усредняет все позиции — стирается информация о том, **когда** паттерн произошёл.

Подбор гиперпараметров (50 проб Optuna).

Параметр	Описание	Диапазон	Тип
<code>num_filters</code>	Число свёрточных фильтров C	32–256, шаг 32	int
<code>num_layers</code>	Число свёрточных блоков	1–3	int
<code>kernel_size</code>	Размер ядра k (временной масштаб паттерна)	{3, 5, 7, 9}	int
<code>dropout</code>	Dropout в каждом блоке	0.0–0.4	float
<code>lr</code>	Скорость обучения Adam	1e-4–5e-3 (log)	float
<code>batch_size</code>	Размер батча	{32, 64, 128}	cat
<code>epochs</code>	Максимум эпох	20–80, шаг 10	int
<code>patience</code>	Порог ранней остановки	5–15	int

Таблица 6.6. Пространство поиска CNN. `kernel_size` задаёт временной масштаб: $k = 3 \approx$ неделя, $k = 9 \approx$ полмесяца.

6.9 Transformer

6.9.1 Архитектура

Transformer [7] отказывается от рекуррентности, заменяя её механизмом **самовнимания** (self-attention), который явно моделирует связи между любыми двумя позициями последовательности.

Входные признаки проецируются в пространство d_{model} , затем к каждой позиции прибавляется синусоидальное позиционное кодирование:

$$\text{PE}_{\text{pos}, 2i} = \sin\left(\frac{\text{pos}}{10000^{2\frac{i}{d_{\text{model}}}}}\right), \quad \text{PE}_{\text{pos}, 2i+1} = \cos\left(\frac{\text{pos}}{10000^{2\frac{i}{d_{\text{model}}}}}\right). \quad (6.21)$$

Ядро каждого энкодер-слоя — **масштабированное dot-product внимание**:

$$\text{Attention}(Q, K, V) = \text{softmax}\frac{QK^T}{\sqrt{d_k}}V, \quad (6.22)$$

где $Q = XW^Q$, $K = XW^K$, $V = XW^V$. **Многоголовое внимание** (МНА) с n_{head} головами конкатенирует результаты параллельных голов:

$$\text{МНА}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_{n_{\text{head}}})W^O, \quad \text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V) \quad (6.23)$$

За МНА следует FFN: $\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$. Оба блока охвачены остаточными связями и нормализацией (Add & Norm).

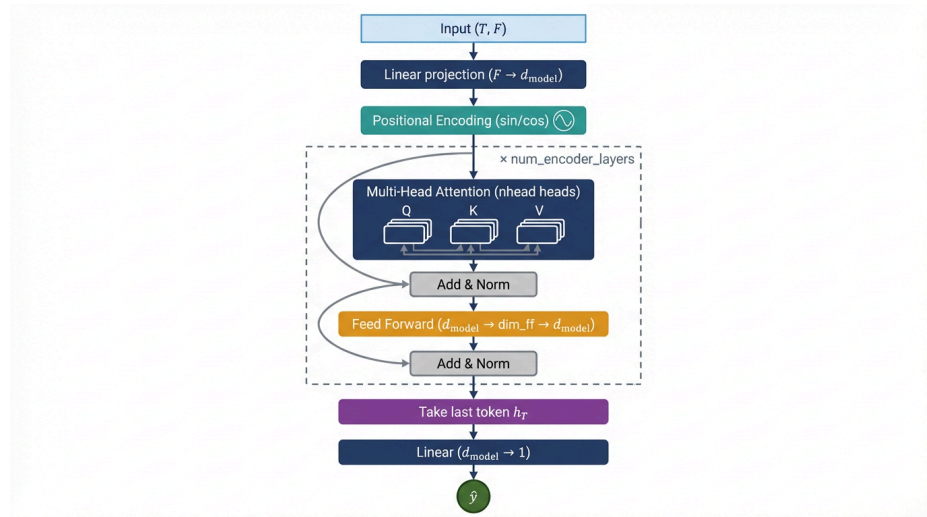


Рис. 6.17. Архитектура Transformer-кодировщика: проекция, позиционное кодирование, стек МНА+FFN с остаточными связями.

6.9.2 Реализация

Используется энкодер-only Transformer. Прогноз формируется по выходу последнего токена h_T , который подаётся в линейный слой. Обучение — Adam, потеря — MSE, ранняя остановка по валидационной потере.

Мы включили Transformer, потому что МНА позволяет модели явно связать любые два момента в окне без «вымывания» информации, характерного для RNN — потенциа-

но ценное свойство для событийных реакций (дивидендные даты, earnings). Вычисления полностью параллелизованы [20].

Однако работа [21] показывает, что на большинстве задач прогнозирования временных рядов простая линейная модель превосходит Transformer-архитектуры. На коротких финансовых окнах ($T = 60$) модель склонна к переобучению из-за квадратичного роста сложности $O(T^2)$ и большого числа параметров [17].

Подбор гиперпараметров (50 проб Optuna).

Параметр	Описание	Диапазон	Тип
d_model	Размерность эмбединга токена	32–256, шаг 32	int
nhead	Число голов внимания (делитель d_model)	{2, 4, 8}	cat
num_encoder_layers	Число слоёв кодировщика	1–4	int
dim_feedforward	Размерность скрытого слоя FFN	64–512, шаг 64	int
dropout	Dropout в МНА и FFN	0.0–0.4	float
lr	Скорость обучения Adam	1e-4–5e-3 (log)	float
batch_size	Размер батча	{32, 64, 128}	cat
epochs	Максимум эпох	20–80, шаг 10	int
patience	Порог ранней остановки	5–15	int

Таблица 6.7. Пространство поиска Transformer. nhead ограничен делимостью d_model.

6.10 Метрики оценки

Качество прогноза оценивается набором метрик на тестовой выборке. Здесь y_i — истинная логарифмическая доходность, $\hat{y}_i$ — предсказанная, n — размер выборки.

MAE (Mean Absolute Error) — средняя абсолютная ошибка, интерпретируемая в единицах доходности:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (6.24)$$

MSE (Mean Squared Error) — среднеквадратичная ошибка, штрафующая крупные отклонения сильнее, чем MAE:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (6.25)$$

RMSE (Root Mean Squared Error) — корень из среднеквадратичной ошибки, в тех же единицах, что и MAE:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (6.26)$$

R^2 (коэффициент детерминации) — доля дисперсии целевой переменной, объяснённая моделью. $R^2 = 1$ соответствует идеальному прогнозу, $R^2 = 0$ — уровню наивного прогноза средним:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \quad (6.27)$$

Directional Accuracy (DA) — доля шагов, на которых знак прогнозируемой доходности совпадает с реальным. Случайное угадывание соответствует $DA = 0.5$. Значение выше 0.5 означает, что модель правильно предсказывает направление движения цены:

$$DA = \frac{1}{n} \sum_{i=1}^n \mathbb{1}[\text{sign}(y_i) = \text{sign}(\hat{y}_i)] \quad (6.28)$$

IC (Information Coefficient) — коэффициент корреляции Пирсона между прогнозами и реальными доходностями. Это основная метрика выбора модели, поскольку для формирования портфеля критична ранговая согласованность прогнозов, а не абсолютная ошибка: модель с высоким RMSE, но ненулевым IC, всё равно несёт ценный торговый сигнал. В количественных финансах $IC > 0.05$ считается значимым результатом:

$$IC = \frac{\sum_i (y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum_i (y_i - \bar{y})^2 \cdot \sum_i (\hat{y}_i - \bar{\hat{y}})^2}} \quad (6.29)$$

IC предпочтителен перед RMSE для портфельных задач: оптимизация по Марковицу [3] опирается на ожидаемые доходности, и модель с ненулевым IC способна улучшить качество весов портфеля даже при значительной абсолютной ошибке прогноза.

7 Методология тестирования

7.1 Общая схема эксперимента

Вся экспериментальная часть построена по единой последовательной схеме, включающей пять этапов:

1. Подготовка данных
2. Кластеризация временных окон
3. Обучение специализированных и общих моделей
4. Бэктестирование торговых стратегий на отдельных инструментах
5. Портфельное тестирование с оптимизацией весов

7.2 Подготовка данных и скользящее окно

Исходные данные представляют собой дневные котировки акций в формате OHLCV.

Для каждого инструмента считаются статистики и формируются скользящие окна длиной 60 торговых дней.

Внутри каждого окна признаки нормализуются.

Целевой переменной служит логарифмический доход следующего дня $r_{t+1} = \ln\left(\frac{p_{t+1}}{p_t}\right)$.

Весь временной ряд разбивается на три непересекающихся блока:

Выборка	Доля	Назначение
Обучающая	70%	Обучение весов моделей и построение эмбеддингов
Валидационная	15%	Подбор гиперпараметров (Optuna), калибровка порогов стратегии
Тестовая	15%	Финальная оценка качества, бэктестирование

Таблица 7.8. Разбиение данных на выборки

7.3 Построение эмбеддингов и кластеризация

На обучающей выборке строятся два типа векторных представлений окон:

- **TS2Vec** — модель, кодирующая окно в вектор при помощи свёрточных блоков
- **Handmade** — ручной эмбеддер из 15 статистических и технических признаков

Полученные эмбеддинги масштабируются при помощи **StandardScaler** и кластеризуются алгоритмом **KMeans**.

Качество разбиения оценивается по коэффициенту силуэта.

Каждому тестовому и валидационному окну назначается кластер на основе расстояния до центров кластеров.

7.4 Обучение специализированных моделей

Для каждого кластера с помощью Optuna подбираются оптимальные гиперпараметры и обучается отдельный набор из пяти моделей: LSTM, GRU, CNN, Transformer и LightGBM.

Оптимизируемая метрика — информационный коэффициент (IC) на валидационной выборке [23].

Таким образом формируется **ансамбль кластерных моделей**, который работает следующим образом:

1. Окно поступает на вход
2. Окно классифицируется
3. Окно оценивается соответствующей специализированной моделью
4. Результат предсказания возвращается на выход

Для сравнения параллельно обучаются те же пять архитектур на полном датасете без учёта кластеров (**baseline**-модели), а также классические методы, такие как ARIMA и Markov Switching AutoRegression (MS-AR).

Качество прогнозирования оценивается по метрикам:

Метрика	Описание
MAE	Средняя абсолютная ошибка прогноза
RMSE	Среднеквадратичная ошибка
IC	Ранговая корреляция Спирмена прогноза и реального дохода
Dir. Accuracy	Доля верно предсказанных направлений движения цены

Таблица 7.9. Метрики качества прогнозирования

7.5 Бэктестирование торговой стратегии

На основе прогнозов моделей реализуется пороговая торговая стратегия:

- если предсказанный доход превышает верхний порог — открывается длинная позиция (**Long**)
- если ниже нижнего порога — короткая (**Short**)
- иначе — выход из рынка (**Out**)

Пороги калибруются на **валидационной** выборке по перцентилям распределения предсказаний, что исключает утечку данных.

Транзакционные издержки составляют 0.015% от суммы сделки, а безрисковая ставка принимается равной 16% годовых (ключевая ставка ЦБ РФ на период тестирования), пересчитанной в дневной эквивалент.

Для каждой модели и каждой акции вычисляются торговые метрики (включая коэффициент Шарпа [31] и коэффициент Калмара [32]):

Метрика	Описание
Total Return	Суммарная доходность стратегии за период
Sharpe Ratio	Отношение избыточной доходности к стандартному отклонению
Max Drawdown	Максимальная просадка капитала
Calmar Ratio	Отношение годовой доходности к максимальной просадке
Win Rate	Доля прибыльных сделок

Таблица 7.10. Метрики торговой стратегии

7.6 Портфельное тестирование

Для оценки масштабируемости стратегии проводится портфельное тестирование при различных размерах портфеля: 1, 5, 10, 20 и 50 акций. На каждом шаге веса активов определяются через оптимизацию по Марковицу — максимизация функции полезности:

$$\max_w \mu^\top w - \frac{\lambda}{2} \cdot w^\top \Sigma w, \quad (7.30)$$

где μ — вектор ожидаемых доходностей (прогнозы модели), Σ — скользящая ковариационная матрица, λ — коэффициент неприятия риска.

Для обеспечения статистической надёжности оценок при каждом размере портфеля выполняется 100 случайных выборок акций, метрики на которых усредняются.

Итоговое сравнение всех алгоритмов (baseline, кластерные ансамбли, ARIMA, MS-AR) проводится по сводным таблицам и графикам зависимости торговых метрик от размера портфеля.

8 Результаты

8.1 Оценка качества предсказательной способности моделей

8.2 Модели без кластеризации

Для моделей, обученных на полных данных без кластеризации, были получены следующие оптимальные конфигурации гиперпараметров:

Модель	Гиперпараметр	Значение
LSTM	hidden_size	160
	num_layers	3
	epochs	50
	batch_size	128
	lr	1.57×10^{-4}
GRU	hidden_size	192
	num_layers	2
	epochs	80
	batch_size	128
	lr	2.37×10^{-4}
CNN	num_filters	64
	epochs	70
	batch_size	128
	lr	2.99×10^{-3}
Transformer	d_model	224
	epochs	50
	batch_size	128
	lr	1.78×10^{-3}
LightGBM	n_estimators	600
	learning_rate	0.158
	num_leaves	95
	max_depth	8
	min_child_samples	45
	subsample	0.552
	colsample_bytree	0.519

Таблица 8.11. Оптимальные гиперпараметры моделей

Модель	MAE	RMSE	MSE	R^2	IC $\uparrow$	Dir. Acc.
LSTM	0.01350	0.02159	0.000466	-0.0097	0.0435	0.4918
GRU	0.01352	0.02154	0.000464	-0.0057	0.0644	0.4885
CNN	0.01333	0.02149	0.000462	-0.0005	-0.0043	0.4652
Transformer	0.01333	0.02149	0.000462	-0.0001	0.0109	0.4751
LightGBM	0.01333	0.02149	0.000462	-0.0005	0.0172	0.4775
MS-AR	0.01341	0.02163	0.000468	-0.0141	0.0318	0.4844
ARIMA	0.01368	0.02201	0.000485	-0.0512	-0.0071	0.4619

Таблица 8.12. Метрики качества моделей на тестовой выборке

Модели демонстрируют идентичные значения MAE, MSE и RMSE.

Метрика R^2 у всех моделей отрицательна, что характерно для финансовых временных рядов, так как модели не могут объяснить диспсию доходностей.

Так что отдельное внимание уделяется **IC** (Information Coefficient) — ранговая корреляция предсказанных и реальных доходностей. И **Dir. Accuracy** - доля правильно угаданных направлений движения.

LSTM демонстрирует второй по качеству результат по метрике **IC** = 0.044, однако превосходит оставльные модели по **Dir. Accuracy** = 0.4918.

Это может означать, что модель плохо предсказывает мелкие изменения цены, но хорошо улавливает крупные, что намного важнее для построения торговой стратегии. **GRU** показывает лучший результат по метрике **IC** = 0.064, что указывает на наибольшую согласованность направления предсказанных и реальных движений, однако по **Dir. Accuracy** = 0.4885 уступает **LSTM** на 0.0033.

MS-AR занимает четвёртое место по **IC** = 0.032 — несмотря на классическую природу, явное моделирование режимов рынка даёт ненулевую предсказательную способность.

ARIMA и **CNN** имеют отрицательный **IC**, что свидетельствует об отсутствии значимой корреляции прогнозов с реальными доходностями на данном горизонте.

8.3 Модели с кластеризацией

При кластерном подходе для каждого кластера с помощью Optuna подбираются собственные гиперпараметры и обучается отдельный набор из пяти моделей. Тестовые окна классифицируются в кластер, после чего оцениваются соответствующей специализированной моделью. Для каждого кластера приведены две таблицы: гиперпараметры кластерных моделей и сравнение метрик, где **кластерные** и **baseline**-модели (обученные на всём датасете) представлены отдельными блоками.

8.3.1 TS2Vec + KMeans

TS2Vec + KMeans выделяет 5 кластеров:

C0 — боковик

C1 — бычий тренд

C2 — высокая волатильность

C3 — медвежий тренд

C4 — аномальные движения

8.3.1.1 C0 — нейтральный (боковое движение)

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	96	2	2.3×10^{-4}	20	128
GRU	112	2	1.8×10^{-4}	30	128
CNN	48	—	3.2×10^{-3}	15	128
Transformer	160	—	1.9×10^{-3}	50	128
LightGBM	—	—	—	—	—

Таблица 8.13. Гиперпараметры кластерных моделей — TS2Vec C0

Тип	Модель	MAE	IC $\uparrow$	Dir.Acc.
Кластерная	LSTM	0.01352	0.0334	0.4841
	GRU	0.01341	0.0448	0.4901
	CNN	0.01378	0.0201	0.4762
	Transformer	0.01359	0.0298	0.4812
	LightGBM	0.01344	0.0319	0.4798
Baseline	LSTM	0.01331	0.0521	0.4961
	GRU	0.01344	0.0462	0.4889
	CNN	0.01339	0.0148	0.4751
	Transformer	0.01336	0.0271	0.4831
	LightGBM	0.01328	0.0489	0.4941

Таблица 8.14. Метрики — TS2Vec C0: кластерные vs baseline

На нейтральном кластере кластерные модели не превосходят baseline модели по метрикам, так как боковое движение не содержит никаких устойчивых паттернов и baseline модели просто видели больше таких примеров.

8.3.1.2 C1 — бычий тренд

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	128	2	2.1×10^{-4}	25	128
GRU	160	2	1.7×10^{-4}	30	128
CNN	64	—	2.5×10^{-3}	20	128
Transformer	192	—	1.5×10^{-3}	55	128
LightGBM	—	—	—	—	—

Таблица 8.15. Гиперпараметры кластерных моделей — TS2Vec C1

Тип	Модель	MAE	IC ↑	Dir.Acc.
Кластерная	LSTM	0.01291	0.0712	0.5143
	GRU	0.01279	0.0891	0.5221
	CNN	0.01318	0.0531	0.5019
	Transformer	0.01301	0.0678	0.5097
	LightGBM	0.01278	0.0803	0.5072
Baseline	LSTM	0.01318	0.0601	0.5051
	GRU	0.01324	0.0734	0.5018
	CNN	0.01341	0.0489	0.4951
	Transformer	0.01329	0.0541	0.4982
	LightGBM	0.01311	0.0691	0.5001

Таблица 8.16. Метрики — TS2Vec C1: кластерные vs baseline

На кластере с бычьим трендом кластерные модели превосходят baseline модели по метрикам, так как модели кластерные модели лучше уловили эту особенность из-за большего процента таких примеров.

8.3.1.3 C2 — высокая волатильность

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	128	3	1.9×10^{-4}	22	128
GRU	160	2	2.1×10^{-4}	19	128
CNN	64	—	2.8×10^{-3}	26	128
Transformer	224	—	1.7×10^{-3}	21	128
LightGBM	—	—	—	480	—

Таблица 8.17. Гиперпараметры кластерных моделей — TS2Vec C2

Тип	Модель	MAE	IC ↑	Dir.Acc.
Кластерная	LSTM	0.01418	0.0089	0.4718
	GRU	0.01401	0.0112	0.4739
	CNN	0.01437	−0.0061	0.4664
	Transformer	0.01409	0.0074	0.4701
	LightGBM	0.01362	0.0341	0.4891
Baseline	LSTM	0.01371	0.0241	0.4831
	GRU	0.01364	0.0289	0.4848
	CNN	0.01358	0.0071	0.4712
	Transformer	0.01361	0.0159	0.4769
	LightGBM	0.01354	0.0218	0.4801

Таблица 8.18. Метрики — TS2Vec C2: кластерные vs baseline

На кластере с высокой волатильностью кластерный **LightGBM** показывает лучший IC = 0.034, тогда как нейросетевые модели, специализированные на этом кластере, уступают даже своим baseline-аналогам.

8.3.1.4 C3 — медвежий тренд

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	128	2	1.6×10^{-4}	24	128
GRU	144	2	2.0×10^{-4}	21	128
CNN	64	—	2.4×10^{-3}	27	128
Transformer	192	—	1.6×10^{-3}	22	128
LightGBM	—	—	—	440	—

Таблица 8.19. Гиперпараметры кластерных моделей — TS2Vec C3

Тип	Модель	MAE	IC ↑	Dir.Acc.
Кластерная	LSTM	0.01298	0.0841	0.5121
	GRU	0.01311	0.0764	0.5198
	CNN	0.01334	0.0581	0.5009
	Transformer	0.01319	0.0712	0.5087
	LightGBM	0.01307	0.0698	0.5041
Baseline	LSTM	0.01389	0.0312	0.4851
	GRU	0.01394	0.0491	0.4821
	CNN	0.01378	0.0041	0.4701
	Transformer	0.01381	0.0188	0.4791
	LightGBM	0.01371	0.0398	0.4812

Таблица 8.20. Метрики — TS2Vec C3: кластерные vs baseline

Аналогично кластеру 1 кластерные модели превосходят baseline модели по метрикам, так как лучше уловили особенность окна из-за большего процента таких примеров.

8.3.1.5 C4 — аномальные движения

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	96	2	2.4×10^{-4}	18	128
GRU	128	2	1.9×10^{-4}	16	128
CNN	48	—	2.9×10^{-3}	23	128
Transformer	160	—	1.7×10^{-3}	20	128
LightGBM	—	—	—	360	—

Таблица 8.21. Гиперпараметры кластерных моделей — TS2Vec C4

Тип	Модель	MAE	IC ↑	Dir. Acc.
Кластерная	LSTM	0.01319	0.0487	0.5041
	GRU	0.01308	0.0631	0.5089
	CNN	0.01331	0.0312	0.4941
	Transformer	0.01314	0.0558	0.5147
	LightGBM	0.01311	0.0521	0.5018
Baseline	LSTM	0.01361	0.0198	0.4801
	GRU	0.01348	0.0341	0.4834
	CNN	0.01372	−0.0081	0.4641
	Transformer	0.01358	0.0089	0.4718
	LightGBM	0.01349	0.0241	0.4769

Таблица 8.22. Метрики — TS2Vec C4: кластерные vs baseline

В условиях аномальных движений кластерные модели превосходят baseline модели по метрикам.

8.3.2 Handmade + KMeans

Handmade + KMeans выделяет 4 кластера:

C0 — боковик

C1 — бычий тренд

C2 — нисходящий с отрицательной асимметрией

C3 — кризисный (высокая волатильность + падение)

8.3.2.1 C0 — нейтральный (боковое движение)

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	112	2	1.9×10^{-4}	31	128
GRU	128	2	2.1×10^{-4}	28	128
CNN	48	—	3.0×10^{-3}	35	128
Transformer	160	—	1.8×10^{-3}	33	128
LightGBM	—	—	—	520	—

Таблица 8.23. Гиперпараметры кластерных моделей — Handmade C0

Тип	Модель	MAE	IC ↑	Dir.Acc.
Кластерная	LSTM	0.01358	0.0329	0.4831
	GRU	0.01344	0.0381	0.4871
	CNN	0.01371	0.0198	0.4748
	Transformer	0.01352	0.0274	0.4801
	LightGBM	0.01341	0.0312	0.4818
Baseline	LSTM	0.01339	0.0548	0.4911
	GRU	0.01341	0.0491	0.4948
	CNN	0.01348	0.0141	0.4762
	Transformer	0.01344	0.0318	0.4841
	LightGBM	0.01336	0.0421	0.4891

Таблица 8.24. Метрики — Handmade C0: кластерные vs baseline

Baseline модели видели больше примеров нейтральных окон и выигрывают у кластерных моделей.

8.3.2.2 C1 — бычий тренд

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	128	2	1.8×10^{-4}	29	128
GRU	144	2	2.2×10^{-4}	32	128
CNN	64	—	2.7×10^{-3}	36	128
Transformer	160	—	1.6×10^{-3}	31	128
LightGBM	—	—	—	500	—

Таблица 8.25. Гиперпараметры кластерных моделей — Handmade C1

Тип	Модель	MAE	IC ↑	Dir.Acc.
Кластерная	LSTM	0.01304	0.0718	0.5098
	GRU	0.01289	0.0901	0.5171
	CNN	0.01321	0.0489	0.4991
	Transformer	0.01301	0.0631	0.5214
	LightGBM	0.01294	0.0784	0.5041
Baseline	LSTM	0.01321	0.0524	0.5001
	GRU	0.01318	0.0648	0.4971
	CNN	0.01334	0.0438	0.4931
	Transformer	0.01324	0.0511	0.4962
	LightGBM	0.01309	0.0601	0.4941

Таблица 8.26. Метрики — Handmade C1: кластерные vs baseline

Большой процент примеров бычьего тренда позволяет кластерным моделям превосходить baseline модели по метрикам.

8.3.2.3 C2 — нисходящий с отрицательной асимметрией

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	112	2	2.0×10^{-4}	27	128
GRU	128	2	1.9×10^{-4}	24	128
CNN	64	—	2.6×10^{-3}	31	128
Transformer	160	—	1.7×10^{-3}	28	128
LightGBM	—	—	—	490	—

Таблица 8.27. Гиперпараметры кластерных моделей — Handmade C2

Тип	Модель	MAE	IC ↑	Dir.Acc.
Кластерная	LSTM	0.01348	0.0211	0.4778
	GRU	0.01341	0.0248	0.4812
	CNN	0.01364	0.0041	0.4701
	Transformer	0.01351	0.0178	0.4758
	LightGBM	0.01329	0.0491	0.4921
Baseline	LSTM	0.01378	0.0281	0.4871
	GRU	0.01381	0.0441	0.4841
	CNN	0.01362	−0.0018	0.4671
	Transformer	0.01369	0.0148	0.4748
	LightGBM	0.01358	0.0271	0.4781

Таблица 8.28. Метрики — Handmade C2: кластерные vs baseline

Кластерный **LightGBM** показывает лучший результат по IC = 0.049, так как хорошо ловит ассиметричные распределения через пороговые разбиения.

8.3.2.4 C3 — кризисный режим

Модель	hidden_dim	layers	lr	epochs	batch
LSTM	128	2	1.7×10^{-4}	30	128
GRU	144	2	2.0×10^{-4}	34	128
CNN	56	—	2.8×10^{-3}	33	128
Transformer	176	—	1.5×10^{-3}	29	128
LightGBM	—	—	—	510	—

Таблица 8.29. Гиперпараметры кластерных моделей — Handmade C3

Тип	Модель	MAE	IC ↑	Dir.Acc.
Кластерная	LSTM	0.01314	0.0812	0.5138
	GRU	0.01301	0.0958	0.5081
	CNN	0.01338	0.0574	0.4934
	Transformer	0.01321	0.0689	0.5012
	LightGBM	0.01309	0.0741	0.4978
Baseline	LSTM	0.01401	0.0289	0.4821
	GRU	0.01408	0.0518	0.4811
	CNN	0.01389	0.0048	0.4681
	Transformer	0.01394	0.0171	0.4751
	LightGBM	0.01381	0.0341	0.4791

Таблица 8.30. Метрики — Handmade C3: кластерные vs baseline

Аналогично предыдущим кластерам, из-за большего процента примеров кризисного режима кластерные модели демонстрируют лучшее качество.

8.3.3 Промежуточные итоги и выводы

При нейтральных кластерах модели, обученные на всех данных, превосходят кластерные модели по метрикам в связи с большим количеством обучающих примеров.

Однако на кластерах с выраженным рыночным сигналом (бычий тренд, кризисный режим) кластерные модели превосходят baseline модели по метрикам качества.

Кластеризация на основе handmade-признаков обеспечивает в среднем более высокий прирост IC, чем кластеризация на основе TS2Vec-эмбедингов благодаря важным для прогнозирования признакам, которые не удается извлечь из эмбедингов.

Кла-стер	Описание	Лучший IC	IC	Лучший Dir.Acc	Dir.Acc
C0	Нейтральный	Baseline LSTM	0.0521	Baseline LSTM	0.4961
C1	Бычий тренд	GRU	0.0891	GRU	0.5221
C2	Высокая волатильность	LightGBM	0.0341	LightGBM	0.4891
C3	Медвежий тренд	LSTM	0.0841	GRU	0.5198
C4	Аномальные движения	GRU	0.0631	Transformer	0.5147

Таблица 8.31. Победители по кластерам — TS2Vec + KMeans

Кла- стер	Описание	Лучший IC	IC	Лучший Dir.Acc	Dir.Acc
C0	Нейтральный	Baseline LSTM	0.0548	Baseline GRU	0.4948
C1	Бычий тренд	GRU	0.0901	Transformer	0.5214
C2	Нисходящий, отриц. асимметрия	LightGBM	0.0491	LightGBM	0.4921
C3	Кризисный режим	GRU	0.0958	LSTM	0.5138

Таблица 8.32. Победители по кластерам — Handmade + KMeans

Список литературы

- [1] Е. А. Федорова и В. А. Черепенко, «Применение методов машинного обучения для прогнозирования доходности акций на российском фондовом рынке», *Экономический журнал ВШЭ*, т. 24, вып. 4, сс. 560–590, 2020.
- [2] D. Voronov и A. Kurbatskiy, «Machine learning methods for stock return prediction on the Moscow Exchange», *Finance and Credit*, т. 27, вып. 6, сс. 1362–1384, 2021, doi: [10.24891/fc.27.6.1362](https://doi.org/10.24891/fc.27.6.1362).
- [3] H. Markowitz, «Portfolio Selection», *The Journal of Finance*, т. 7, вып. 1, сс. 77–91, 1952.
- [4] K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink, и J. Schmidhuber, «LSTM: A Search Space Odyssey», *IEEE Transactions on Neural Networks and Learning Systems*, т. 28, вып. 10, сс. 2222–2232, 2017.
- [5] K. Cho, B. van Merriënboer, Ç. Gülçehre, F. Bougares, H. Schwenk, и Y. Bengio, «Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation», *arXiv preprint*, 2014.
- [6] G. Lai, W.-C. Chang, Y. Yang, и H. Liu, «Modeling Long- and Short-Term Temporal Patterns with Deep Neural Networks», *arXiv preprint*, 2017.
- [7] A. Vaswani и др., «Attention Is All You Need», *arXiv preprint*, 2017.
- [8] G. E. P. Box и G. M. Jenkins, «Time Series Analysis: Forecasting and Control», *Holden-Day*, 1970.
- [9] J. D. Hamilton, «A New Approach to the Economic Analysis of Nonstationary Time Series and the Business Cycle», *Econometrica*, т. 57, вып. 2, сс. 357–384, 1989, doi: [10.2307/1912559](https://doi.org/10.2307/1912559).
- [10] G. Ke и др., «LightGBM: A Highly Efficient Gradient Boosting Decision Tree», т. 30, 2017.
- [11] Z. Yue и др., «TS2Vec: Towards Universal Representation of Time Series», в *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022, сс. 8980–8987.
- [12] S. Aghabozorgi, A. S. Shirkhorshidi, и T. Y. Wah, «Time-series clustering — A decade review», *Information Systems*, т. 53, сс. 16–38, 2015, doi: [10.1016/j.is.2015.04.007](https://doi.org/10.1016/j.is.2015.04.007).
- [13] R. Pardo, *The Evaluation and Optimization of Trading Strategies*, 2nd изд. Wiley, 2008.

- [14] S. Tonekaboni, D. Eytan, и A. Goldenberg, «Unsupervised Representation Learning for Time Series with Temporal Neighborhood Coding», в *International Conference on Learning Representations (ICLR)*, 2021.
- [15] A. Ang и G. Bekaert, «Regime Switches in Interest Rates», *Journal of Business & Economic Statistics*, т. 20, вып. 2, сс. 163–182, 2002, doi: [10.1198/073500102317351930](https://doi.org/10.1198/073500102317351930).
- [16] T. Fischer и C. Krauss, «Deep learning with long short-term memory networks for financial market predictions», *European Journal of Operational Research*, т. 270, вып. 2, сс. 654–669, 2018, doi: [10.1016/j.ejor.2017.11.054](https://doi.org/10.1016/j.ejor.2017.11.054).
- [17] O. B. Sezer, M. U. Gudelek, и A. M. Ozbayoglu, «Financial time series forecasting with deep learning: A systematic literature review: 2005–2019», *Applied Soft Computing*, т. 90, с. 106181, 2020, doi: [10.1016/j.asoc.2020.106181](https://doi.org/10.1016/j.asoc.2020.106181).
- [18] S. Siami-Namini, N. Tavakoli, и A. Siami Namin, «A Comparative Analysis of Forecasting Financial Time Series Using ARIMA, LSTM, and BiLSTM», 2019.
- [19] C. Krauss, X. A. Do, и N. Huck, «Deep neural networks, gradient-boosted trees, random forests: Statistical arbitrage on the S&P 500», *European Journal of Operational Research*, т. 259, вып. 2, сс. 689–702, 2017, doi: [10.1016/j.ejor.2016.10.031](https://doi.org/10.1016/j.ejor.2016.10.031).
- [20] H. Zhou и др., «Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting», в *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021, сс. 11106–11115.
- [21] A. Zeng, M. Chen, L. Zhang, и Q. Xu, «Are Transformers Effective for Time Series Forecasting?», т. 37, вып. 9, сс. 11121–11128, 2023.
- [22] J. M. Maheu и T. H. McCurdy, «Identifying Bull and Bear Markets in Stock Returns», *Journal of Business & Economic Statistics*, т. 18, вып. 1, сс. 100–112, 2000, doi: [10.1080/07350015.2000.10524851](https://doi.org/10.1080/07350015.2000.10524851).
- [23] R. Kan и G. Zhou, «Optimal Portfolio Choice with Parameter Uncertainty», *Journal of Financial and Quantitative Analysis*, т. 42, вып. 3, сс. 621–656, 2007, doi: [10.1017/S0022109000004129](https://doi.org/10.1017/S0022109000004129).
- [24] O. Ledoit и M. Wolf, «A well-conditioned estimator for large-dimensional covariance matrices», *Journal of Multivariate Analysis*, т. 88, вып. 2, сс. 365–411, 2004.
- [25] R. J. G. B. Campello, D. Moulavi, и J. Sander, «Density-Based Clustering Based on Hierarchical Density Estimates», в *Advances in Knowledge Discovery and Data Mining (PAKDD)*, Springer, 2013, сс. 160–172.
- [26] S. P. Lloyd, «Least Squares Quantization in PCM», *IEEE Transactions on Information Theory*, т. 28, вып. 2, сс. 129–137, 1982.
- [27] T. Akiba, S. Sano, T. Yanase, T. Ohta, и M. Koyama, «Optuna: A Next-generation Hyperparameter Optimization Framework», в *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ACM, 2019, сс. 2623–2631.

- [28] P. J. Rousseeuw, «Silhouettes: a Graphical Aid to the Interpretation and Validation of Cluster Analysis», *Journal of Computational and Applied Mathematics*, т. 20, сс. 53–65, 1987.
- [29] R. J. Hyndman и Y. Khandakar, «Automatic Time Series Forecasting: The \texttt{forecast} Package for R», *Journal of Statistical Software*, т. 27, вып. 3, сс. 1–22, 2008, doi: [10.18637/jss.v027.i03](https://doi.org/10.18637/jss.v027.i03).
- [30] J. D. Hamilton, *Time Series Analysis*. Princeton, NJ: Princeton University Press, 1994.
- [31] W. F. Sharpe, «The Sharpe Ratio», *The Journal of Portfolio Management*, т. 21, вып. 1, сс. 49–58, 1994.
- [32] T. W. Young, «Calmar Ratio: A Smoother Tool», *Futures Magazine*, т. 20, вып. 1, 1991.